

Machine Learning Notes

Manish Giri

June 2026

Contents

1	Introduction	1
1.1	Concepts of Artificial Intelligence	1
1.1.1	Key Components of AI	1
1.1.2	Types of AI	2
1.2	Why Machine Learning?	2
1.3	Problems Machine Learning Can Solve	2
1.3.1	Supervised Learning	2
1.3.2	Unsupervised Learning	3
1.3.3	Data Representation in Machine Learning	4
1.4	Knowing Your Task and Knowing Your Data	4
1.4.1	Key Questions to Consider	4
1.4.2	The Bigger Picture	5
1.5	Why Python?	5
1.6	Important Python Libraries for Machine Learning	6
1.6.1	NumPy	6
1.6.2	Pandas	6
1.6.3	Matplotlib	7
1.6.4	Seaborn	7
1.6.5	Scikit-learn	8
1.6.6	TensorFlow	9
1.6.7	PyTorch	10
1.6.8	OpenCV	10
1.6.9	NLTK	10
1.7	Types of Machine Learning Models	11
1.8	Linear Algebra for Machine Learning	11
1.8.1	Scalars	11
1.8.2	Vectors	12
1.8.3	Matrices	12
1.8.4	Matrix Multiplication	12
1.8.5	Importance in Machine Learning	12
1.9	Probability Theory	13
1.9.1	Probability	13
1.9.2	Random Variables	13

1.9.3	Conditional Probability	14
1.9.4	Bayes' Theorem	14
1.9.5	Importance in Machine Learning	14
1.10	Statistics	14
1.10.1	Mean	15
1.10.2	Median	15
1.10.3	Mode	15
1.10.4	Variance	15
1.10.5	Standard Deviation	17
1.10.6	Importance in Machine Learning	18
1.11	Information Theory	19
1.11.1	What is Information?	19
1.11.2	Entropy	20
1.11.3	Entropy Diagram	21
1.11.4	Information Gain	21
1.11.5	Decision Tree Diagram using Information Gain	22
1.11.6	Cross Entropy	22
1.11.7	Kullback-Leibler (KL) Divergence	23
1.11.8	Mutual Information	24
1.11.9	Importance of Information Theory in Machine Learning	24
1.11.10	Summary	25
1.12	Optimization Theory	25
1.12.1	Objective Function	25
1.12.2	Gradient Descent	25
1.12.3	Types of Gradient Descent	26
1.12.4	Momentum Optimizer	28
1.12.5	AdaGrad	28
1.12.6	RMSProp	29
1.12.7	Adam Optimizer	29
1.12.8	Comparison of Optimization Algorithms	30
1.13	Convex Functions	30
1.13.1	Mathematical Definition	30
1.13.2	Example of Convex Function	30
1.13.3	Non-Convex Function	31
1.13.4	Convex vs Non-Convex Functions	32
1.13.5	Importance in Machine Learning	32
1.14	Loss Function	33
1.14.1	Mean Squared Error (MSE)	33
1.14.2	Mean Absolute Error (MAE)	33
1.14.3	Binary Cross Entropy	34
1.14.4	Categorical Cross Entropy	34
1.15	Loss Function Example	34

1.16	Overfitting vs Underfitting	35
1.16.1	Underfitting	35
1.16.2	Overfitting	36
1.16.3	Good Fit	37
1.16.4	Bias-Variance Tradeoff	37
2	Data Preprocessing	38
2.1	Introduction	38
2.1.1	Need for Data Preprocessing	38
2.1.2	Data Preprocessing Process	39
2.2	Data Cleaning	39
2.2.1	Objectives	39
2.2.2	Missing Values	40
2.2.3	Noisy Data	41
2.2.4	Outlier Detection	42
2.2.5	Data Consistency	43
2.3	Data Integration	44
2.3.1	Objectives of Data Integration	44
2.3.2	Schema Integration	44
2.3.3	Redundancy Detection	45
2.4	Data Transformation	48
2.4.1	Objectives of Data Transformation	48
2.4.2	Normalization	48
2.4.3	Attribute Construction	49
2.4.4	Discretization	50
2.4.5	Concept Hierarchy Generation	51
2.4.6	Advantages of Data Transformation	52
2.5	Data Reduction	52
2.5.1	Objectives of Data Reduction	53
2.5.2	Data Cube Aggregation	53
2.5.3	Attribute Subset Selection	54
2.5.4	Numerosity Reduction	55
2.5.5	Dimensionality Reduction	56
2.5.6	Data Compression	57
2.5.7	Advantages of Data Reduction	58
2.6	Data Wrangling	58
2.7	Data Visualization	60
2.7.1	Exploratory Data Analysis (EDA)	61
3	Supervised Learning	63
3.1	Classification	64
3.1.1	Use-Cases:	64

3.2	Regression	64
3.2.1	Use-Cases:	65
3.3	Supervised Learning Algorithms	65
3.3.1	Linear Regression:	65
3.3.2	K-Nearest Neighbors:	66
3.3.3	Decision Trees:	69
3.3.4	Naive Bayes:	70
3.3.5	Logistic Regression:	73
3.3.6	Support Vector Machines(SVM):	74
3.3.7	Random Forest:	76
4	Ensemble Modeling	78
4.1	Bagging (Bootstrap Aggregation)	78
4.1.1	Algorithm	78
4.1.2	Mathematical Idea	78
4.1.3	Example	78
4.1.4	Architecture	79
4.2	Boosting	79
4.2.1	Algorithm	79
4.2.2	Mathematical Idea	79
4.2.3	Example	79
4.2.4	Architecture	80
4.3	Stacking	80
4.3.1	Algorithm	80
4.3.2	Architecture	80
5	Unsupervised Learning	81
5.1	Key Terminologies	81
5.2	Clustering Algorithms	81
5.2.1	K-Means Clustering	81
5.2.2	DBSCAN (Density-Based Spatial Clustering of Applications with Noise)	86
5.2.3	Gaussian Mixture Model (GMM)	89
5.2.4	Hierarchical Clustering	93
5.3	Dimensionality Reduction	97
5.3.1	Principal Component Analysis (PCA)	97
5.3.2	t-Distributed Stochastic Neighbor Embedding (t-SNE)	99
5.4	Generative Models	100
5.4.1	Autoencoders (AE)	100
5.4.2	Variational Autoencoders (VAE)	102

6	Model Evaluation	104
6.1	Evaluation of Supervised Learning Models	104
6.1.1	Confusion Matrix	104
6.1.2	Accuracy	104
6.1.3	Precision	104
6.1.4	Recall	104
6.1.5	F1-Score	104
6.1.6	Specificity	104
6.1.7	Mean Squared Error (MSE)	105
6.1.8	Root Mean Squared Error (RMSE)	105
6.1.9	Mean Absolute Error (MAE)	105
6.1.10	Coefficient of Determination (R^2)	105
6.2	Evaluation of Unsupervised Learning Models	105
6.2.1	Silhouette Score	105
6.2.2	Davies-Bouldin Index (DBI)	106
6.2.3	Calinski-Harabasz Index (CHI)	106
6.2.4	Within Cluster Sum of Squares (WCSS)	106
6.2.5	Reconstruction Error	107
7	Machine Learning Pipelines	108
7.1	Overview of Machine Learning Pipeline	108
7.2	Data Collection	109
7.2.1	Sources of Data	109
7.2.2	Objectives	109
7.3	Data Preprocessing	110
7.3.1	Handling Missing Values	110
7.3.2	Normalization	110
7.3.3	Standardization	111
7.4	Feature Engineering	111
7.4.1	Techniques	111
7.4.2	Principal Component Analysis (PCA)	111
7.5	Data Splitting	112
7.5.1	Typical Split Ratios	112
7.6	Model Training	112
7.6.1	Loss Function	112
7.7	Model Evaluation	113
7.7.1	Classification Metrics	113
7.7.2	Regression Metrics	113
7.8	Hyperparameter Tuning	113
7.8.1	Grid Search	113
7.8.2	Random Search	113
7.9	Model Deployment	114

7.9.1	Deployment Methods	114
7.10	Monitoring and Maintenance	114
7.10.1	Monitoring Metrics	114
7.10.2	Retraining	114
7.11	Advantages of ML Pipelines	115
8	Working with Text Data	116
9	Deep Learning Basics	117
10	Case Studies and Applications	118

Chapter 1

Introduction

Machine learning is about extracting knowledge from data. It is a research field at the intersection of statistics, artificial intelligence, and computer science and is also known as predictive analytics or statistical learning. The application of machine learning methods has in recent years become ubiquitous in everyday life. From automatic recommendations of which movies to watch, to what food to order or which products to buy, to personalized online radio and recognizing our friends in our photos, many modern websites and devices have machine learning algorithms at their core. When we look at a complex website like Facebook, Amazon, or Netflix, it is very likely that every part of the site contains multiple machine learning models.

Outside of commercial applications, machine learning has had a tremendous influence on the way data-driven research is done today. The tools introduced in this book have been applied to diverse scientific problems such as understanding stars, finding distant planets, discovering new particles, analyzing DNA sequences, and providing personalized cancer treatments.

Our application doesn't need to be as large-scale or world-changing as these examples in order to benefit from machine learning, though. In this chapter, we will explain why machine learning has become so popular and discuss what kinds of problems can be solved using machine learning. Then, we will show you how to build our first machine learning model, introducing important concepts along the way.

1.1 Concepts of Artificial Intelligence

Artificial intelligence is the simulation of human intelligence in machines, enabling them to learn, reason, perceive, solve complex problems, and make decisions to perform tasks that typically require human interaction.

1.1.1 Key Components of AI

- a. **Learning:** The ability to learn from data and improve performance.

- b. **Reasoning:** The capability to make decisions and draw conclusions based on available information.
- c. **Problem Solving:** The process of finding optimal solutions to complex tasks or challenges.
- d. **Perception:** The ability to interpret information from the environment, such as images, sounds, and sensor data.
- e. **Language Understanding:** The capability to comprehend, interpret, and generate human language (NLP).

1.1.2 Types of AI

- i. **Narrow AI (Weak AI):** AI designed to perform specific tasks, such as voice assistants and recommendation systems.
- ii. **General AI (Strong AI):** A hypothetical AI capable of performing any intellectual task that a human can do, such as Human-Like Robot (learn, reason, plan).
- iii. **Super AI:** A theoretical form of AI that surpasses human intelligence in all aspects, such as JARVIS, ULTRON.

1.2 Why Machine Learning?

Early AI systems relied on hand-coded if-else rules created by human experts. While effective for simple tasks like spam filtering, these systems were difficult to modify and required extensive domain knowledge. They also struggled with complex problems such as face detection in images. Machine Learning overcomes these limitations by learning patterns automatically from large amounts of data, enabling accurate decision-making without explicitly programmed rules.

1.3 Problems Machine Learning Can Solve

Machine Learning (ML) automates decision-making by learning patterns from data instead of relying on manually written rules. ML algorithms can be broadly classified into **Supervised Learning** and **Unsupervised Learning**.

1.3.1 Supervised Learning

In supervised learning, the algorithm learns from **input-output pairs** provided by a user. The goal is to predict the correct output for new, unseen inputs. Since the desired outputs are known, the algorithm is guided by a “teacher.” **Examples:**

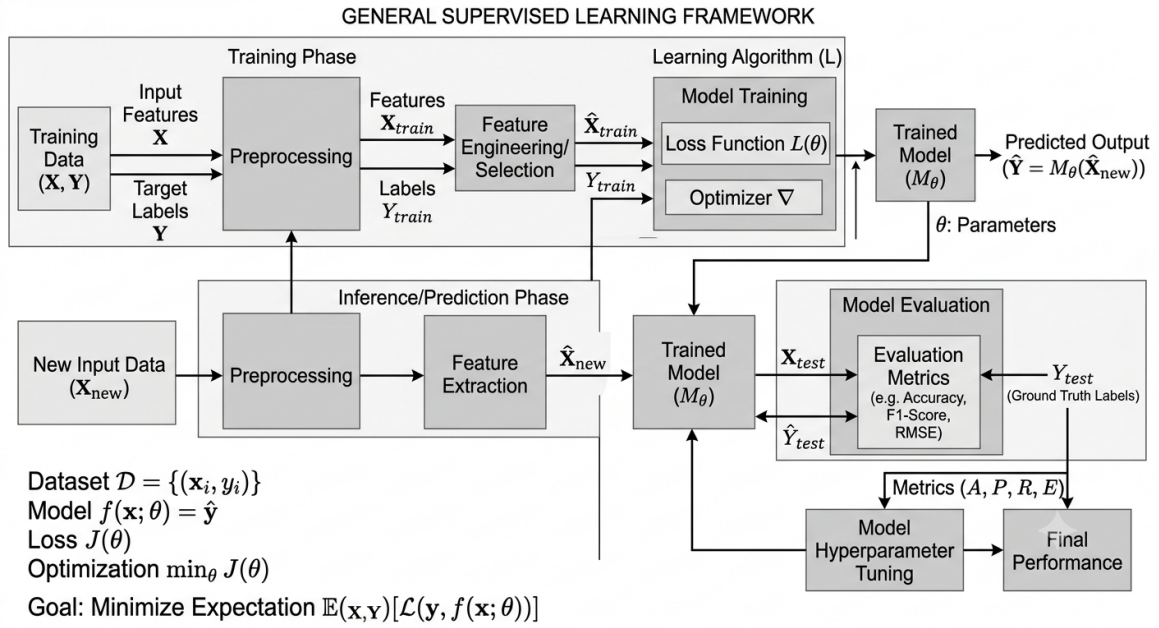


Figure 1.1: Workflow of Supervised Machine Learning

- **Spam Detection:** Classifying emails as spam or non-spam.
- **Handwritten Digit Recognition:** Identifying zip codes from handwritten envelopes.
- **Medical Diagnosis:** Determining whether a tumor is benign or malignant from medical images.
- **Fraud Detection:** Detecting fraudulent credit card transactions.

Supervised learning is effective when a dataset with known outcomes is available.

1.3.2 Unsupervised Learning

In unsupervised learning, only the input data is provided, and there are **no predefined outputs**. The algorithm discovers hidden patterns, structures, or relationships within the data. **Examples:**

- **Topic Identification:** Finding common themes in a collection of documents or blog posts.
- **Customer Segmentation:** Grouping customers based on similar preferences or behaviors.
- **Anomaly Detection:** Identifying unusual website access patterns that may indicate bugs or security threats.

Unsupervised learning is useful for exploring data and discovering unknown patterns.

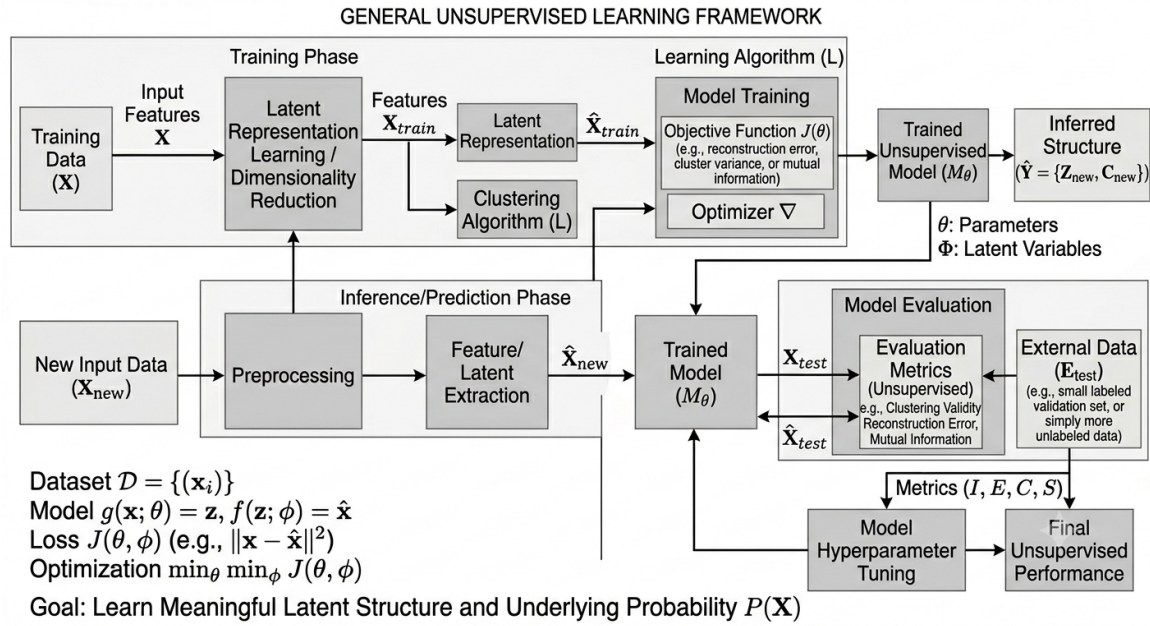


Figure 1.2: Workflow of Unsupervised Machine Learning

1.3.3 Data Representation in Machine Learning

Machine learning works with data organized as a table:

- **Rows** represent individual data points, called **samples**.
- **Columns** represent characteristics of the data, called **features**.

For example, a customer dataset may include features such as age, gender, and purchase history. The quality of these features greatly affects model performance. The process of selecting or creating useful features is called **Feature Engineering** or **Feature Extraction**.

A machine learning model can only learn from the information provided. If important features are missing, the model may not be able to make accurate predictions.

1.4 Knowing Your Task and Knowing Your Data

Understanding the problem and the data is one of the most important steps in Machine Learning. Before selecting an algorithm, it is necessary to understand the dataset and how it relates to the task being solved. Choosing an algorithm without understanding the data may lead to poor results.

1.4.1 Key Questions to Consider

- What problem or question am I trying to solve?

- Can the available data answer this question?
- How can the problem be formulated as a Machine Learning task?
- Is there enough data to represent the problem accurately?
- Are the selected features useful for making predictions?
- How will the success of the model be measured?
- How will the Machine Learning solution fit into the overall system or business process?

1.4.2 The Bigger Picture

Machine Learning algorithms are only one part of the overall problem-solving process. While building models, it is important to focus on the actual problem being solved rather than only the technical details. A complex model is not useful if it does not address the right problem.

When developing Machine Learning solutions, always keep the final objective in mind and be aware of any assumptions made about the data, features, and problem domain. Understanding the broader context helps ensure that the model provides meaningful and practical results.

1.5 Why Python?

Python is widely used in Machine Learning because it is easy to learn, has a simple and readable syntax, and provides powerful libraries such as NumPy, Pandas, Scikit-learn, TensorFlow, and PyTorch. It also offers strong community support, cross-platform compatibility, and rapid development capabilities, making it the preferred language for Machine Learning applications.

Reasons for Using Python in Machine Learning:

- **Easy to Learn and Use:** Simple syntax makes coding and debugging easier.
- **Extensive Libraries:** Libraries such as NumPy, Pandas, Scikit-learn, TensorFlow, and PyTorch simplify data analysis and model development.
- **Large Community Support:** A large community provides tutorials, documentation, and solutions to common problems.
- **Cross-Platform Compatibility:** Python programs can run on Windows, Linux, and macOS.
- **Rapid Development:** Allows quick prototyping and testing of machine learning models.

- **Integration Capability:** Easily integrates with databases, web applications, and other programming languages.
- **Open Source:** Python and most of its Machine Learning libraries are free to use.

1.6 Important Python Libraries for Machine Learning

1.6.1 NumPy

NumPy (Numerical Python) is the fundamental library for numerical computing in Python. It provides support for multidimensional arrays, matrices, and mathematical operations.

Uses:

- Array and matrix operations
- Mathematical computations
- Linear algebra
- Statistical calculations

Example:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
print(arr.mean())
```

Output:

3.0

1.6.2 Pandas

Pandas is a data manipulation and analysis library that provides DataFrame and Series data structures.

Uses:

- Reading CSV and Excel files
- Data cleaning and preprocessing
- Data analysis

Example:

```
import pandas as pd

data = pd.DataFrame({
    "Name": ["John", "Alice"],
    "Age": [20, 22]
})

print(data.head())
```

Output:

	Name	Age
0	John	20
1	Alice	22

1.6.3 Matplotlib

Matplotlib is a data visualization library used for creating graphs and plots.

Uses:

- Line plots
- Bar charts
- Histograms
- Scatter plots

Example:

```
import matplotlib.pyplot as plt

plt.plot([1, 2, 3], [10, 20, 30])
plt.show()
```

Output:

1.6.4 Seaborn

Seaborn is a statistical visualization library built on top of Matplotlib.

Uses:

- Heatmaps
- Distribution plots
- Correlation analysis

Example:

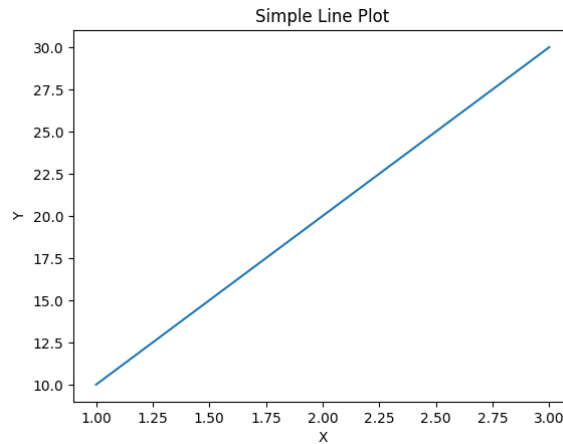


Figure 1.3: Line Plot using Matplotlib

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6],
    'C': [7, 8, 9]
})

plt.figure(figsize=(6, 4))
sns.heatmap(
    df.corr(),
    annot=True,
    cmap="Blues", # Blue to white shades
    linewidths=0.5
)

plt.title("Correlation Heatmap")
plt.savefig("seaborn_heatmap.png", bbox_inches="tight", dpi
    =300)
plt.show()
```

Output:

1.6.5 Scikit-learn

Scikit-learn is a Machine Learning library that provides tools for classification, regression, and clustering.

Uses:

- Classification

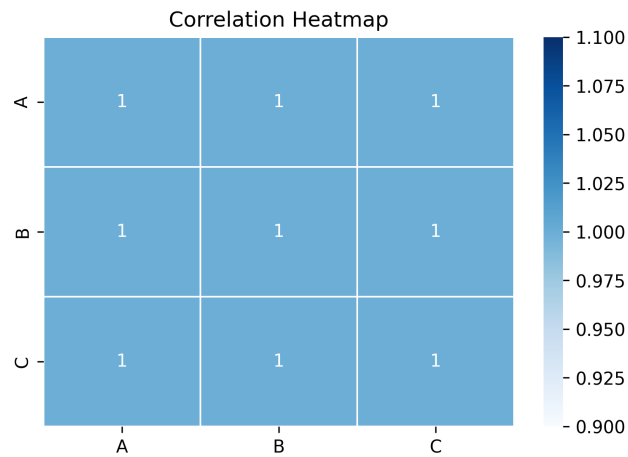


Figure 1.4: Heatmap using Seaborn

- Regression
- Clustering
- Model evaluation

Example:

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
```

Output:

```
LinearRegression()
```

1.6.6 TensorFlow

TensorFlow is an open-source Deep Learning framework used to build neural networks.

Uses:

- Deep Learning
- Neural Networks
- Computer Vision
- NLP

Example:

```
import tensorflow as tf
model = tf.keras.Sequential()
```

Output:

```
<Sequential name=sequential, built=False>
```

1.6.7 PyTorch

PyTorch is a Deep Learning framework widely used in AI research and development.

Uses:

- Deep Learning
- Neural Networks
- Research Projects

Example:

```
import torch
x = torch.tensor([1,2,3])
```

Output:

```
tensor([1, 2, 3])
```

1.6.8 OpenCV

OpenCV is used for image and video processing applications.

Uses:

- Image Processing
- Face Detection
- Object Detection

Example:

```
import cv2
image = cv2.imread("image.jpg")
```

Output: Image loaded successfully.

1.6.9 NLTK

NLTK (Natural Language Toolkit) is used for Natural Language Processing tasks.

Uses:

- Text Processing
- Tokenization
- Sentiment Analysis

Example:

```
from nltk.tokenize import word_tokenize
print(word_tokenize("Machine Learning is amazing"))
```

Output:

```
['Machine', 'Learning', 'is', 'amazing', '.']
```

1.7 Types of Machine Learning Models

Machine Learning models can be categorized according to their learning objectives, structure, and mathematical formulation. Each model type has specific strengths suited to particular kinds of data and tasks. Broadly, ML models are classified into six categories: linear models, probabilistic models, decision-based models, clustering models, dimensionality reduction models, and neural networks (Bishop, 2006; Murphy, 2012; Goodfellow et al., 2016).

Model Type	Learning Paradigm	Examples and Applications
Linear Models	Supervised	Linear Regression, Logistic Regression
Probabilistic Models	Supervised / Un-supervised	Naïve Bayes, Hidden Markov Models
Decision-Based Models	Supervised	Decision Trees, Random Forests
Clustering Models	Unsupervised	k-Means, Hierarchical Clustering
Dimensionality Reduction	Unsupervised	PCA, t-SNE
Ensemble Models	Supervised	Bagging, Boosting, XGBoost
Neural Networks	Supervised / Un-supervised	Deep Learning Models for Images, NLP

Table 1.1: Types of Machine Learning Models

1.8 Linear Algebra for Machine Learning

Linear Algebra is the branch of mathematics that deals with vectors, matrices, and linear transformations. It forms the foundation of many Machine Learning algorithms because data is often represented as vectors and matrices.

1.8.1 Scalars

A scalar is a single numerical value.

Example:

$$x = 10$$

Here, 10 is a scalar.

1.8.2 Vectors

A vector is an ordered collection of numbers.

Example:

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

Applications:

- Feature representation
- Input and output data
- Word embeddings

1.8.3 Matrices

A matrix is a rectangular arrangement of numbers.

Example:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

Applications:

- Dataset representation
- Image processing
- Neural network weights

1.8.4 Matrix Multiplication

Matrix multiplication combines information from rows and columns.

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5 \\ 6 \end{bmatrix}$$
$$AB = \begin{bmatrix} 17 \\ 39 \end{bmatrix}$$

1.8.5 Importance in Machine Learning

- Data is represented as matrices.
- Features are represented as vectors.

- Neural networks use matrix operations extensively.
- Dimensionality reduction techniques such as PCA rely on linear algebra.

Sample 1	x_1	x_2	x_3	
Sample 2				

Figure 1.5: Dataset Representation as a Matrix

1.9 Probability Theory

Probability Theory studies uncertainty and randomness. Machine Learning algorithms use probability to make predictions and estimate outcomes.

1.9.1 Probability

Probability measures the likelihood of an event occurring.

$$P(E) = \frac{\text{Number of favorable outcomes}}{\text{Total number of outcomes}}$$

Example:

For a fair die,

$$P(4) = \frac{1}{6}$$

1.9.2 Random Variables

A random variable is a variable whose value depends on the outcome of a random experiment.

Example:

$$X = \text{Outcome of a die roll}$$

Possible values:

$$X = \{1, 2, 3, 4, 5, 6\}$$

1.9.3 Conditional Probability

Conditional probability is the probability of an event occurring given that another event has occurred.

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

1.9.4 Bayes' Theorem

Bayes' Theorem updates probabilities using new evidence.

::contentReference[oaicite:0]index=0

Applications:

- Spam Detection
- Medical Diagnosis
- Recommendation Systems

1.9.5 Importance in Machine Learning

- Naive Bayes Classification
- Hidden Markov Models
- Probabilistic Graphical Models
- Uncertainty Estimation

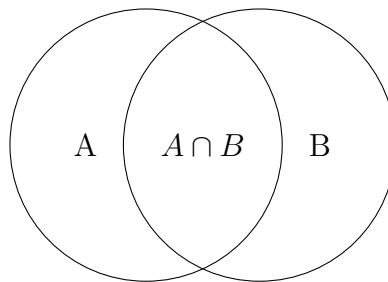


Figure 1.6: Events A and B in Probability

1.10 Statistics

Statistics is the science of collecting, analyzing, and interpreting data. It helps Machine Learning models understand patterns and trends in data.

1.10.1 Mean

Mean is the average value of a dataset.

$$\bar{x} = \frac{\sum_{i=1}^n x_i}{n}$$

Example:

Data:

$$\{2, 4, 6, 8\}$$

Mean:

$$\bar{x} = 5$$

1.10.2 Median

The median is the middle value when data is arranged in order.

$$\{1, 3, 5, 7, 9\}$$

$$\text{Median} = 5$$

1.10.3 Mode

The mode is the most frequently occurring value.

$$\{1, 2, 2, 3, 4\}$$

$$\text{Mode} = 2$$

1.10.4 Variance

Variance is a statistical measure that describes how far data values are spread from the mean (average). A small variance indicates that data points are close to the mean, whereas a large variance indicates that data points are widely dispersed.

Formula

For a population:

$$\sigma^2 = \frac{\sum_{i=1}^N (x_i - \mu)^2}{N} \quad (1.1)$$

For a sample:

$$s^2 = \frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n - 1} \quad (1.2)$$

where:

- σ^2 = Population variance
- s^2 = Sample variance
- x_i = Individual observation
- μ = Population mean
- $\bar{x}$ = Sample mean
- N = Population size
- n = Sample size

Example

Consider the dataset:

$$\{2, 4, 6, 8, 10\}$$

Mean:

$$\bar{x} = \frac{2 + 4 + 6 + 8 + 10}{5} = 6$$

Variance:

$$s^2 = \frac{(2 - 6)^2 + (4 - 6)^2 + (6 - 6)^2 + (8 - 6)^2 + (10 - 6)^2}{4}$$

$$s^2 = \frac{16 + 4 + 0 + 4 + 16}{4} = \frac{40}{4} = 10$$

Graph

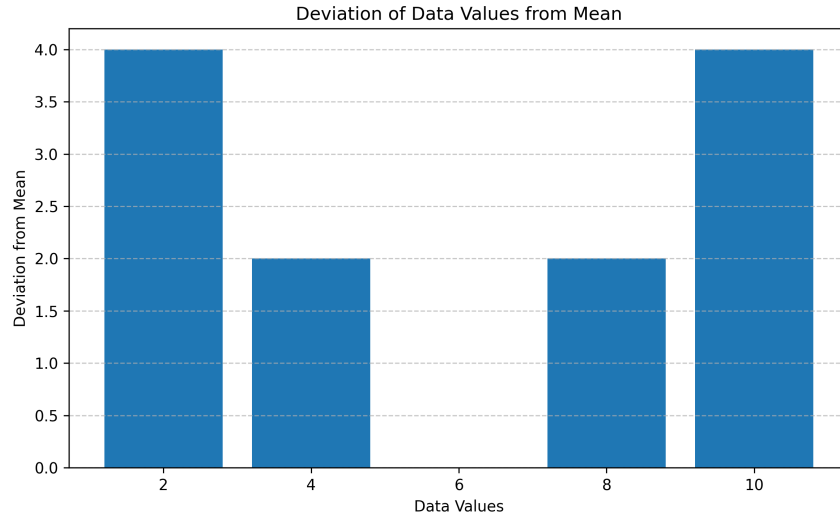


Figure 1.7: Deviation of data values from the mean.

Variance measures the average squared distance of data points from the mean.

1.10.5 Standard Deviation

Standard deviation is the square root of variance. It measures the average amount of variation or dispersion in a dataset and is expressed in the same unit as the original data.

Formula

For a population:

$$\sigma = \sqrt{\frac{\sum_{i=1}^N (x_i - \mu)^2}{N}} \quad (1.3)$$

For a sample:

$$s = \sqrt{\frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n - 1}} \quad (1.4)$$

where:

- σ = Population standard deviation
- s = Sample standard deviation
- μ = Population mean
- $\bar{x}$ = Sample mean

Example

Using the same dataset:

$$\{2, 4, 6, 8, 10\}$$

Variance:

$$s^2 = 10$$

Standard Deviation:

$$s = \sqrt{10}$$

$$s \approx 3.16$$

Thus, the data values are spread approximately 3.16 units from the mean.

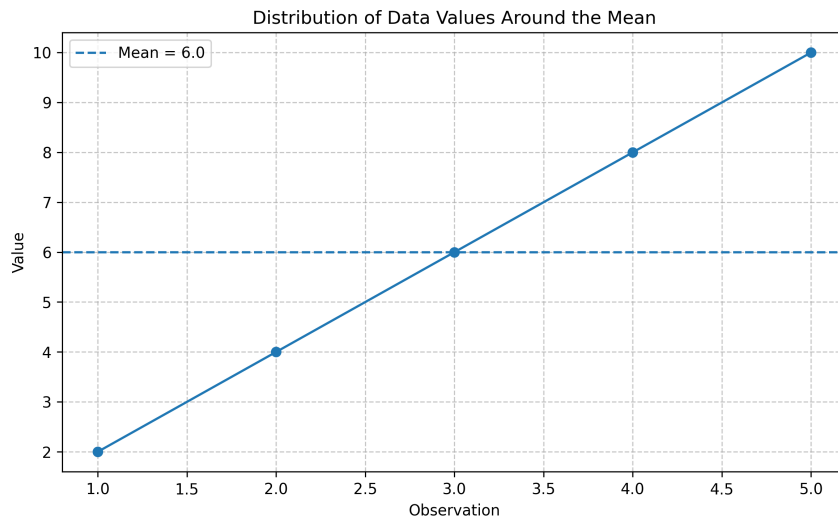
Graph

Figure 1.8: Distribution of data values around the mean.

Standard deviation is widely used in statistics, data mining, and machine learning to measure the spread of data.

1.10.6 Importance in Machine Learning

- Data Analysis
- Feature Engineering
- Outlier Detection

- Model Evaluation

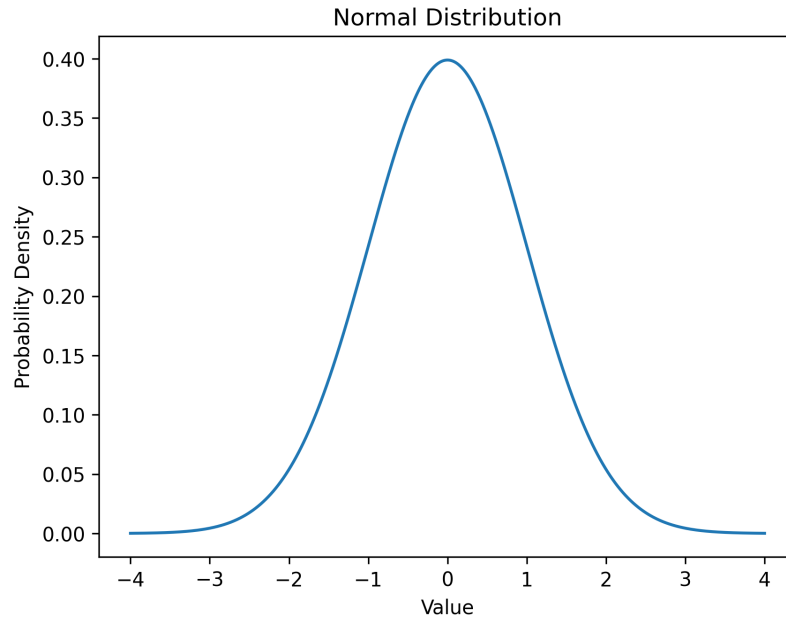


Figure 1.9: Normal Distribution Curve

1.11 Information Theory

Information Theory is a branch of mathematics that studies the quantification, storage, and communication of information. It provides mathematical tools for measuring uncertainty, information content, and similarity between probability distributions. In Machine Learning, Information Theory is widely used in Decision Trees, Feature Selection, Classification, Deep Learning, and Generative Models.

1.11.1 What is Information?

Information refers to the knowledge gained from observing an event. Events that are less likely to occur provide more information than events that are highly predictable.

Example:

- The sun rises in the east. (Low information)
- A fair coin lands on heads. (Higher information)

The information content of an event is given by:

$$I(x) = -\log_2 P(x)$$

where:

- $I(x)$ = Information content
- $P(x)$ = Probability of event x

Example:

If the probability of an event is:

$$P(x) = 0.25$$

Then,

$$I(x) = -\log_2(0.25) = 2 \text{ bits}$$

Thus, rarer events carry more information.

1.11.2 Entropy

Entropy is a measure of uncertainty or randomness in a dataset. It quantifies how unpredictable the outcomes are.

$$H(X) = -\sum_{i=1}^n P(x_i) \log_2 P(x_i)$$

where:

- $H(X)$ = Entropy
- $P(x_i)$ = Probability of outcome x_i

Properties of Entropy

- Entropy is maximum when all outcomes are equally likely.
- Entropy is minimum when one outcome is certain.
- Higher entropy indicates greater uncertainty.

Example: Fair Coin

Consider a fair coin:

$$P(H) = 0.5$$

$$P(T) = 0.5$$

Entropy:

$$H(X) = -(0.5 \log_2 0.5 + 0.5 \log_2 0.5)$$

$$H(X) = 1 \text{ bit}$$

This represents maximum uncertainty.

Example: Biased Coin

$$P(H) = 0.9$$

$$P(T) = 0.1$$

Entropy:

$$H(X) = -(0.9 \log_2 0.9 + 0.1 \log_2 0.1)$$

$$H(X) = 0.469 \text{ bits}$$

Since one outcome is much more likely, uncertainty decreases.

1.11.3 Entropy Diagram

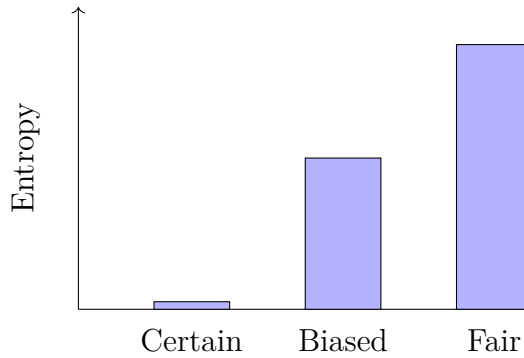


Figure 1.10: Entropy increases as uncertainty increases.

1.11.4 Information Gain

Information Gain measures the reduction in entropy after splitting a dataset.

It is one of the most important concepts used in Decision Tree algorithms such as ID3 and C4.5.

$$IG = H(Parent) - \sum_{i=1}^k \frac{|S_i|}{|S|} H(S_i)$$

where:

- $H(Parent)$ = Entropy before splitting

- $H(S_i)$ = Entropy of subset i
- $|S_i|$ = Number of samples in subset i

Example

Suppose a dataset has entropy:

$$H(Parent) = 1.0$$

After splitting:

$$H(Child) = 0.3$$

Then,

$$IG = 1.0 - 0.3$$

$$IG = 0.7$$

A larger Information Gain indicates a better split.

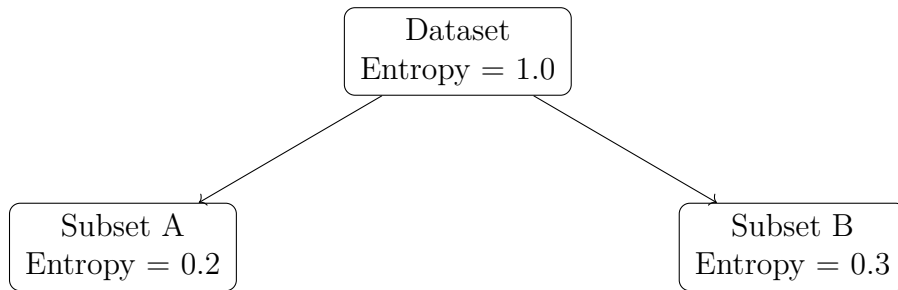
1.11.5 Decision Tree Diagram using Information Gain

Figure 1.11: Entropy reduction after splitting a dataset.

1.11.6 Cross Entropy

Cross Entropy measures the difference between the true probability distribution and the predicted probability distribution.

It is commonly used as a loss function in classification problems.

$$H(P, Q) = - \sum_x P(x) \log_2 Q(x)$$

where:

- $P(x)$ = True distribution

- $Q(x)$ = Predicted distribution

Example

Suppose the true class is:

$$Cat = 1$$

Model prediction:

$$Cat = 0.9$$

$$Dog = 0.1$$

Since the model assigns high probability to the correct class, the Cross Entropy loss will be low.

Applications:

- Neural Networks
- Image Classification
- Natural Language Processing

1.11.7 Kullback-Leibler (KL) Divergence

KL Divergence measures how one probability distribution differs from another.

$$D_{KL}(P||Q) = \sum_x P(x) \log_2 \left(\frac{P(x)}{Q(x)} \right)$$

where:

- $P(x)$ = True distribution
- $Q(x)$ = Approximate distribution

Properties

- KL Divergence is always non-negative.
- KL Divergence equals zero when the distributions are identical.
- Larger values indicate greater differences between distributions.

Applications

- Variational Autoencoders (VAE)
- Generative AI
- Reinforcement Learning
- Distribution Comparison

1.11.8 Mutual Information

Mutual Information measures how much information one random variable provides about another.

$$I(X; Y) = \sum_{x,y} P(x, y) \log_2 \left(\frac{P(x, y)}{P(x)P(y)} \right)$$

Applications:

- Feature Selection
- Dimensionality Reduction
- Dependency Analysis

Higher mutual information indicates stronger dependence between variables.

1.11.9 Importance of Information Theory in Machine Learning

Information Theory is widely used in Machine Learning because it helps quantify uncertainty and improve decision-making.

Applications include:

- Decision Trees using Information Gain
- Feature Selection using Mutual Information
- Neural Networks using Cross Entropy Loss
- Generative Models using KL Divergence
- Probabilistic Modeling
- Data Compression

1.11.10 Summary

Concept	Purpose	ML Application
Information	Measures surprise	Probability Models
Entropy	Measures uncertainty	Decision Trees
Information Gain	Reduces uncertainty	Feature Selection
Cross Entropy	Measures prediction error	Deep Learning
KL Divergence	Compares distributions	Generative AI
Mutual Information	Measures dependency	Feature Selection

Table 1.2: Important Concepts in Information Theory

1.12 Optimization Theory

Optimization Theory is a branch of mathematics that focuses on finding the best solution to a problem by minimizing or maximizing an objective function. In Machine Learning, optimization algorithms are used to find the model parameters that minimize the loss function and improve prediction accuracy.

1.12.1 Objective Function

An objective function (or loss function) measures the error between the predicted output and the actual output.

Example: Mean Squared Error (MSE)

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

where:

- $J(\theta)$ = Cost function
- y_i = Actual value
- $\hat{y}_i$ = Predicted value
- n = Number of samples

The goal is to minimize $J(\theta)$.

1.12.2 Gradient Descent

Gradient Descent is the most widely used optimization algorithm in Machine Learning.

The update rule is:

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

where:

- θ_t = Current parameter value
- θ_{t+1} = Updated parameter value
- η = Learning rate
- $\nabla J(\theta_t)$ = Gradient of the cost function

Working Steps:

1. Initialize parameters randomly.
2. Compute loss.
3. Calculate gradient.
4. Update parameters.
5. Repeat until convergence.

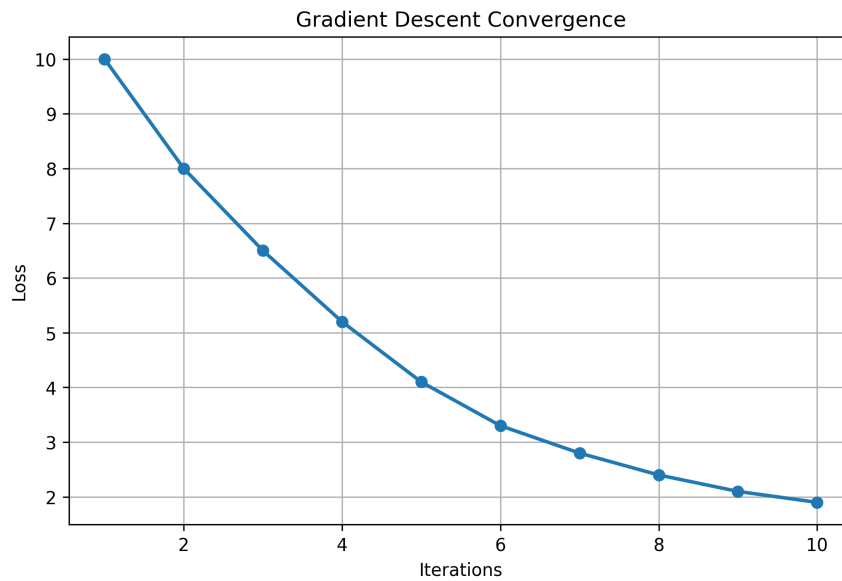


Figure 1.12: Gradient Descent Convergence

1.12.3 Types of Gradient Descent

Batch Gradient Descent

Uses the entire dataset during each update.

$$\theta_{t+1} = \theta_t - \eta \frac{1}{N} \sum_{i=1}^N \nabla J_i(\theta_t)$$

Advantages

- Stable convergence
- Accurate gradient estimation

Disadvantages

- Slow for large datasets

Stochastic Gradient Descent (SGD)

Uses one sample at a time.

$$\theta_{t+1} = \theta_t - \eta \nabla J_i(\theta_t)$$

Advantages

- Faster updates
- Suitable for large datasets

Disadvantages

- Noisy convergence

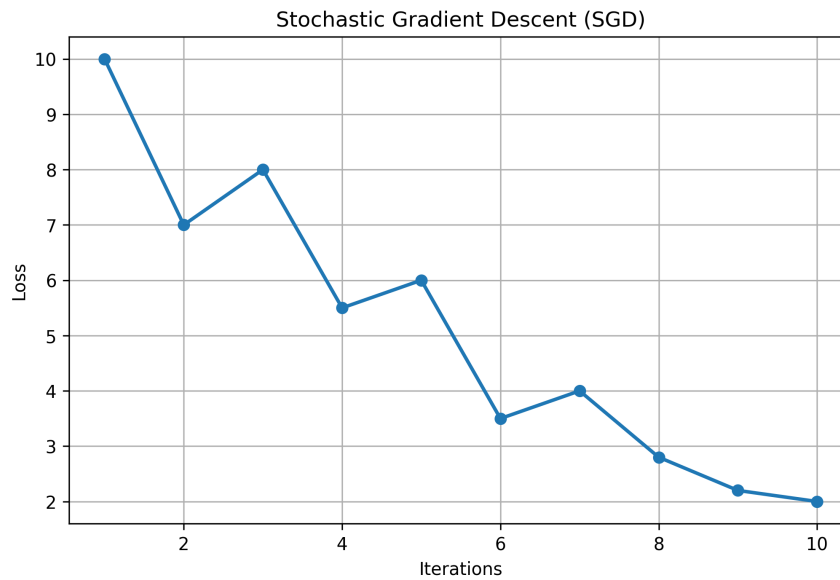


Figure 1.13: SGD Convergence

Mini-Batch Gradient Descent

Uses a small batch of samples.

$$\theta_{t+1} = \theta_t - \eta \frac{1}{m} \sum_{i=1}^m \nabla J_i(\theta_t)$$

where m is the batch size.

Advantages

- Faster than Batch GD
- More stable than SGD

1.12.4 Momentum Optimizer

Momentum accelerates Gradient Descent by considering previous updates.

Velocity:

$$v_t = \beta v_{t-1} + \eta \nabla J(\theta_t)$$

Parameter update:

$$\theta_{t+1} = \theta_t - v_t$$

where:

$$\beta = 0.9$$

is commonly used.

Advantages

- Faster convergence
- Reduces oscillation

1.12.5 AdaGrad

AdaGrad adapts the learning rate for each parameter.

Accumulated Gradient:

$$G_t = G_{t-1} + g_t^2$$

Update Rule:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t + \epsilon}} g_t$$

Advantages

- Good for sparse data

Disadvantages

- Learning rate decreases too much over time

1.12.6 RMSProp

RMSProp solves AdaGrad's diminishing learning rate problem.

$$E[g^2]_t = \beta E[g^2]_{t-1} + (1 - \beta)g_t^2$$

Update:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} g_t$$

Advantages

- Faster convergence
- Effective for RNNs

1.12.7 Adam Optimizer

Adam combines Momentum and RMSProp.

First Moment Estimate:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t$$

Second Moment Estimate:

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)g_t^2$$

Parameter Update:

$$\theta_{t+1} = \theta_t - \eta \frac{m_t}{\sqrt{v_t} + \epsilon}$$

Common values:

$$\beta_1 = 0.9$$

$$\beta_2 = 0.999$$

Advantages

- Fast convergence
- Most popular optimizer in Deep Learning

1.12.8 Comparison of Optimization Algorithms

Optimizer	Speed	Common Usage
Batch GD	Slow	Small Datasets
SGD	Fast	Online Learning
Mini-Batch GD	Very Fast	Deep Learning
Momentum	Fast	CNN Training
AdaGrad	Medium	NLP Tasks
RMSProp	Fast	RNN Training
Adam	Very Fast	Deep Learning

Table 1.3: Comparison of Optimization Algorithms

1.13 Convex Functions

A convex function is a function whose graph always lies below the straight line joining any two points on the graph.

Convex functions are important because optimization algorithms can easily find the global minimum.

1.13.1 Mathematical Definition

A function $f(x)$ is convex if:

$$f(\lambda x_1 + (1 - \lambda)x_2) \leq \lambda f(x_1) + (1 - \lambda)f(x_2)$$

where:

$$0 \leq \lambda \leq 1$$

1.13.2 Example of Convex Function

Consider:

$$f(x) = x^2$$

This function has:

- One global minimum
- No local minima

Global Minimum:

$$x = 0$$

$$f(0) = 0$$

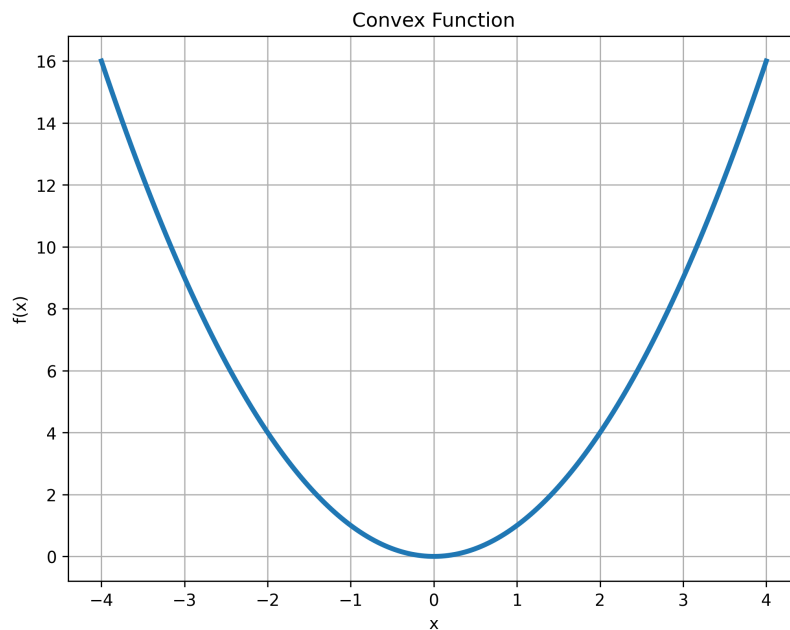


Figure 1.14: Convex Function: $f(x) = x^2$

1.13.3 Non-Convex Function

Consider:

$$f(x) = x^4 - 4x^2$$

This function contains:

- Multiple local minima
- Multiple local maxima

Optimization becomes more difficult because algorithms may get trapped in local minima.

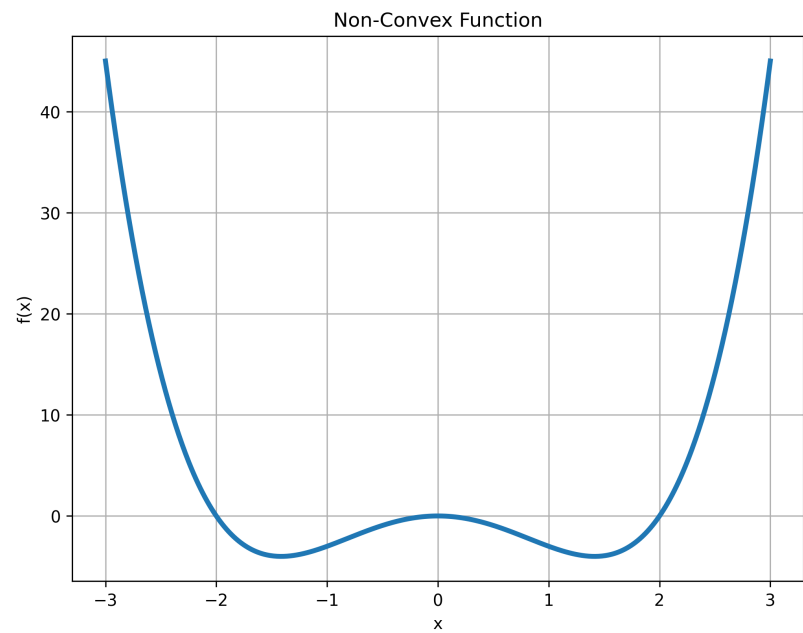


Figure 1.15: Non-Convex Function

1.13.4 Convex vs Non-Convex Functions

Property	Convex	Non-Convex
Shape	Bowl-Shaped	Irregular
Global Minimum	One	Multiple Possible
Optimization Difficulty	Easy	Difficult
Example	x^2	$x^4 - 4x^2$

Table 1.4: Convex vs Non-Convex Functions

1.13.5 Importance in Machine Learning

Convex Functions and Optimization Theory are fundamental in Machine Learning because they help models learn optimal parameters efficiently.

Applications include:

- Linear Regression
- Logistic Regression
- Support Vector Machines (SVM)
- Neural Networks
- Deep Learning Optimization

1.14 Loss Function

A Loss Function measures the difference between the actual output and the predicted output of a Machine Learning model.

The goal of training is to minimize the loss function.

$$Loss = Actual\ Value - Predicted\ Value$$

A lower loss indicates better model performance.

1.14.1 Mean Squared Error (MSE)

MSE calculates the average squared difference between actual and predicted values.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

where:

- y_i = Actual value
- $\hat{y}_i$ = Predicted value
- n = Number of samples

Example

Actual Values:

$$[10, 20, 30]$$

Predicted Values:

$$[12, 18, 28]$$

MSE:

$$\begin{aligned} &= \frac{(10 - 12)^2 + (20 - 18)^2 + (30 - 28)^2}{3} \\ &= \frac{4 + 4 + 4}{3} \\ &= 4 \end{aligned}$$

1.14.2 Mean Absolute Error (MAE)

MAE computes the average absolute difference between actual and predicted values.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

1.14.3 Binary Cross Entropy

Binary Cross Entropy measures the difference between predicted probabilities and actual labels.

$$L = -[y \log(p) + (1 - y) \log(1 - p)]$$

where:

- y = Actual label
- p = Predicted probability

1.14.4 Categorical Cross Entropy

$$Loss = - \sum_{i=1}^C y_i \log(\hat{y}_i)$$

where:

- C = Number of classes
- y_i = True probability
- $\hat{y}_i$ = Predicted probability

1.15 Loss Function Example

Python Code:

```
import numpy as np
import matplotlib.pyplot as plt

epochs = np.arange(1, 21)

loss = [1.8, 1.5, 1.3, 1.1, 1.0,
        0.9, 0.8, 0.7, 0.65, 0.6,
        0.55, 0.5, 0.45, 0.4, 0.35,
        0.3, 0.28, 0.25, 0.22, 0.2]

plt.figure(figsize=(8,5))
plt.plot(epochs, loss, linewidth=3)

plt.xlabel("Epoch")
```

```
plt.ylabel("Loss")
plt.title("Loss Reduction During Training")
plt.grid(True)

plt.savefig(
    "loss_function.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```

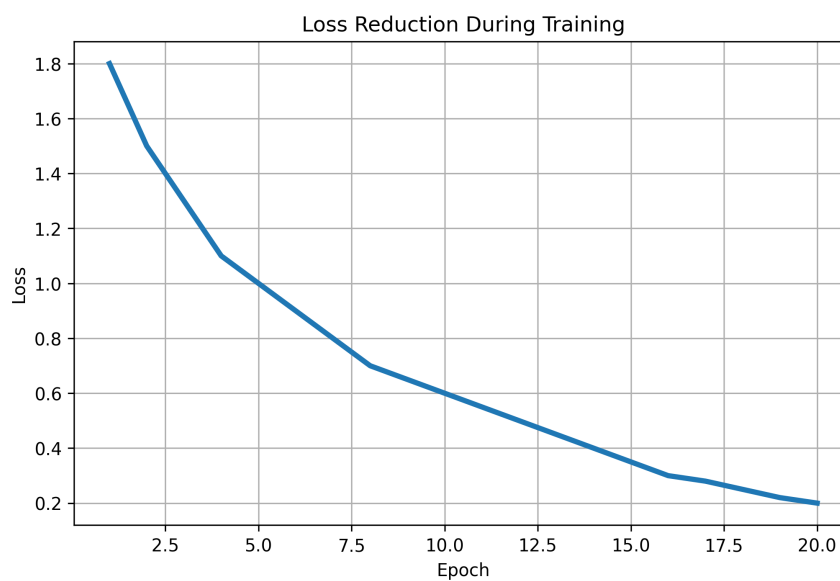


Figure 1.16: Loss Function During Training

1.16 Overfitting vs Underfitting

One of the most important challenges in Machine Learning is achieving a balance between learning and generalization.

A model should learn patterns from training data without memorizing them.

1.16.1 Underfitting

Underfitting occurs when a model is too simple to learn the underlying patterns in data.

Characteristics:

- High Training Error
- High Testing Error

- Poor Performance

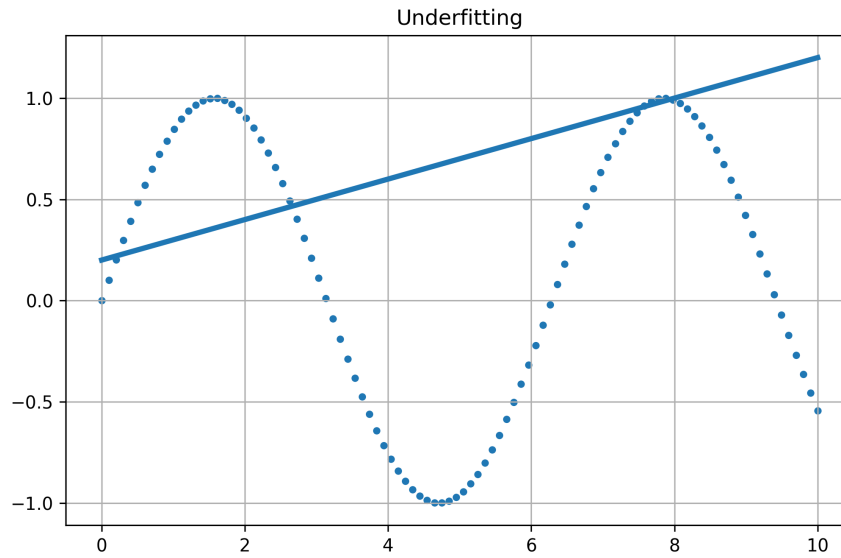


Figure 1.17: Underfitting

1.16.2 Overfitting

Overfitting occurs when a model learns not only useful patterns but also noise from the training data.

Characteristics:

- Low Training Error
- High Testing Error
- Poor Generalization

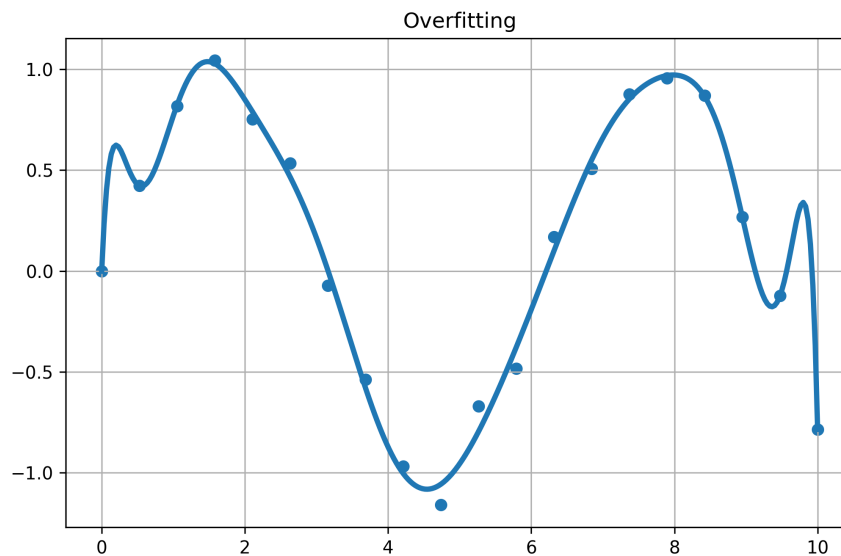


Figure 1.18: Overfitting

1.16.3 Good Fit

A good model balances bias and variance.

Characteristics:

- Low Training Error
- Low Testing Error
- Good Generalization

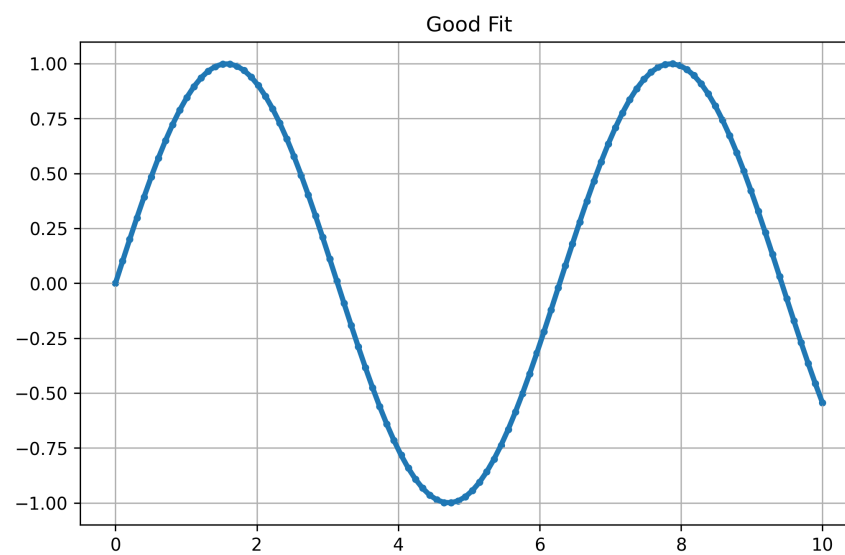


Figure 1.19: Good Fit

1.16.4 Bias-Variance Tradeoff

- Underfitting = High Bias + Low Variance
- Overfitting = Low Bias + High Variance
- Good Fit = Balanced Bias and Variance

Chapter 2

Data Preprocessing

2.1 Introduction

Data preprocessing is a crucial step in data mining and machine learning. Real-world data is often incomplete, noisy, inconsistent, and contains redundant information. Data preprocessing improves data quality and prepares data for efficient analysis.

2.1.1 Need for Data Preprocessing

- Improves data quality.
- Increases mining accuracy.
- Reduces computational complexity.
- Removes inconsistencies.
- Handles missing values and noise.

2.1.2 Data Preprocessing Process

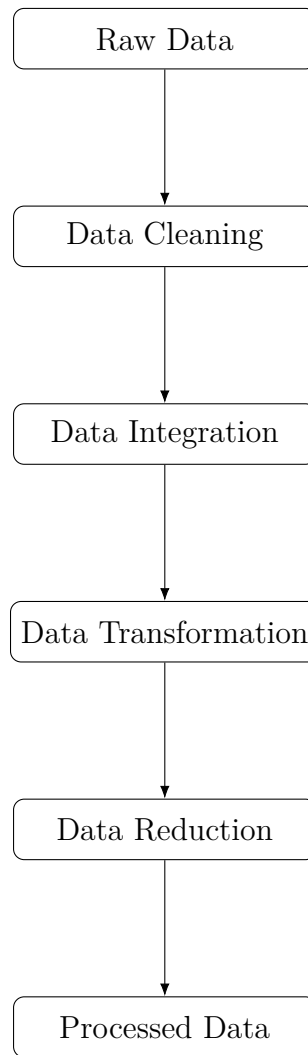


Figure 2.1: Data Preprocessing Workflow

2.2 Data Cleaning

Data cleaning is the process of identifying and correcting incomplete, inaccurate, noisy, inconsistent, and duplicate data.

2.2.1 Objectives

- Handle missing values.
- Remove noise.
- Detect outliers.
- Resolve inconsistencies.

2.2.2 Missing Values

Missing values occur when no value is stored for a particular attribute.

Example

ID	Name	Marks
101	Ram	85
102	Shyam	?
103	Mohan	90

Table 2.1: Dataset containing Missing Values

Methods of Handling Missing Values

1. Ignore Tuple
2. Fill Manually
3. Use Global Constant
4. Mean Imputation
5. Median Imputation
6. Most Probable Value

Mean Imputation

The mean value is calculated as:

$$\bar{x} = \frac{\sum_{i=1}^n x_i}{n}$$

Example:

$$\bar{x} = \frac{80 + 90 + 70}{3} = 80$$

Hence the missing value can be replaced by 80.

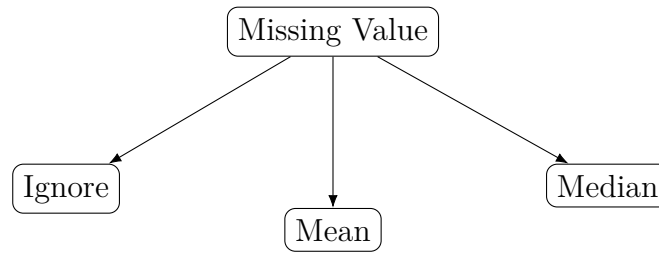
Missing Value Handling Diagram

Figure 2.2: Methods for Handling Missing Values

2.2.3 Noisy Data

Noisy data refers to random errors or variance in a measured variable.

Examples include:

- Typographical errors
- Sensor errors
- Transmission errors

Noise Removal Techniques

1. Binning
2. Regression
3. Clustering

Binning Example

Original Data:

4, 8, 15, 21, 24, 25, 28, 34, 40

Grouped into bins:

[4, 8, 15], [21, 24, 25], [28, 34, 40]

Bin Means:

9, 23.3, 34

Noise Smoothing Diagram

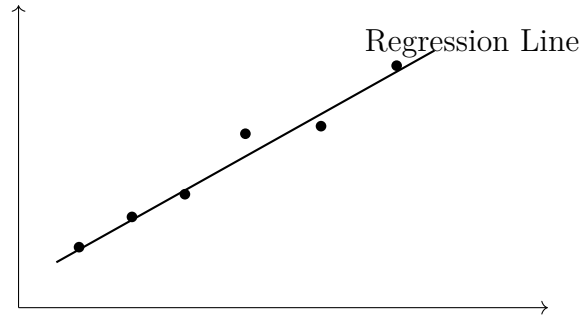


Figure 2.3: Noise Smoothing using Regression

2.2.4 Outlier Detection

Outliers are data objects that significantly deviate from the normal behavior of a dataset.

Examples:

10, 12, 14, 13, 11, 15, 120

Here 120 is an outlier.

Z-Score Method

The Z-score is calculated as:

$$Z = \frac{x - \mu}{\sigma}$$

where

- x = observation
- μ = mean
- σ = standard deviation

If

$$|Z| > 3$$

then the observation is generally considered an outlier.

Standard Deviation Formula

$$\sigma = \sqrt{\frac{\sum_{i=1}^n (x_i - \mu)^2}{n}}$$

Outlier Detection Graph

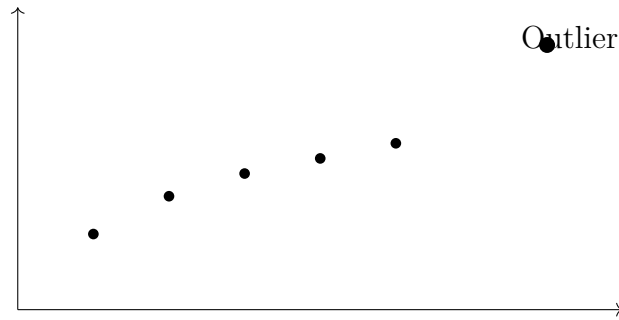


Figure 2.4: Illustration of an Outlier

2.2.5 Data Consistency

Data consistency ensures that data values are logically and semantically correct throughout the dataset.

Examples of Inconsistency

ID	Age	Status
101	-5	Adult
102	25	Child

Table 2.2: Examples of Inconsistent Data

Consistency Checking Rules

- Age must be positive.
- Salary cannot be negative.
- Date of Joining must be before Retirement Date.
- Primary Keys must be unique.

Consistency Verification Process

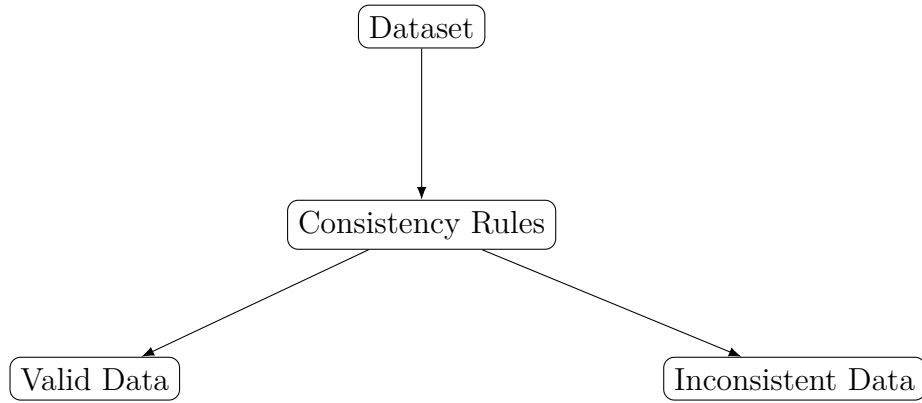


Figure 2.5: Data Consistency Checking Process

2.3 Data Integration

Data Integration is the process of combining data from multiple heterogeneous sources into a single, unified, and consistent dataset. It helps improve data quality, eliminate redundancy, and provide a comprehensive view of the data.

2.3.1 Objectives of Data Integration

- Combine data from multiple sources.
- Eliminate data redundancy.
- Improve data consistency.
- Create a unified data repository.

2.3.2 Schema Integration

Schema Integration is the process of merging schemas from different databases into a unified schema.

Example

Database 1:

StudentID	Name
101	Ram

Table 2.3: Database 1

Database 2:

RollNo	StudentName
101	Ram

Table 2.4: Database 2

After integration:

StudentID	Name
101	Ram

Table 2.5: Unified Schema

In this example:

$$StudentID = RollNo$$

$$Name = StudentName$$

Schema Integration Process

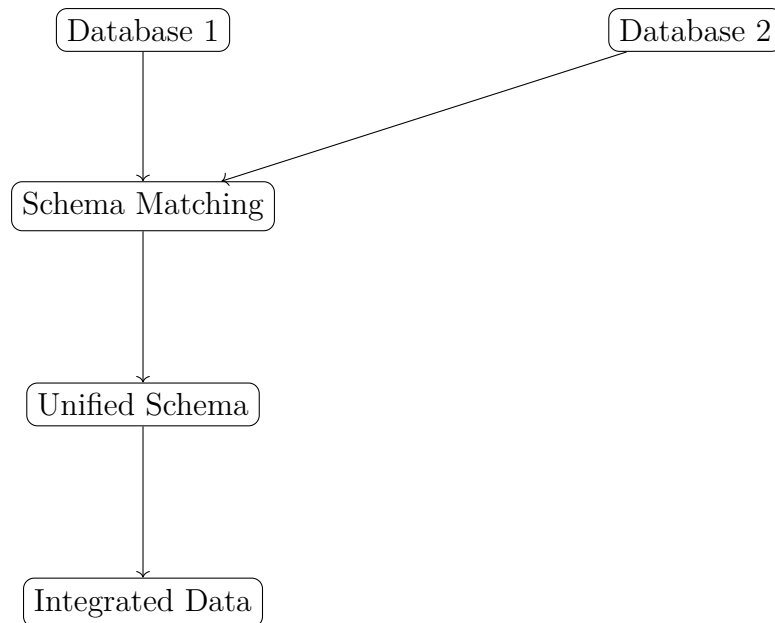


Figure 2.6: Schema Integration Process

2.3.3 Redundancy Detection

Redundancy occurs when the same information is stored multiple times within a dataset. Removing redundant data reduces storage requirements and improves mining efficiency.

Example

StudentID	Date of Birth	Age
101	10/05/2000	25

Table 2.6: Example of Redundant Data

In this example, Age can be calculated from Date of Birth, making the Age attribute redundant.

Correlation Analysis

Redundant attributes are often identified using correlation analysis.

The Pearson Correlation Coefficient is:

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}}$$

where:

- $r = 1$ indicates strong positive correlation.
- $r = -1$ indicates strong negative correlation.
- $r = 0$ indicates no correlation.

If two attributes have a very high correlation, one attribute may be considered redundant.

Correlation Example

Height (cm)	Weight (kg)
150	50
160	58
170	67
180	75
190	84

Table 2.7: Highly Correlated Attributes

Correlation Graph

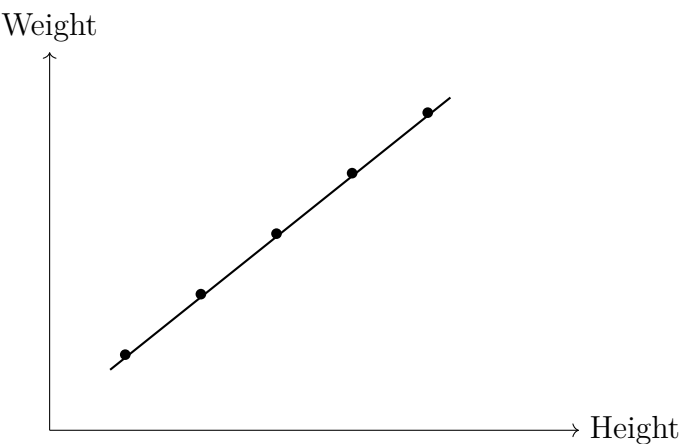


Figure 2.7: Positive Correlation Between Two Attributes

Redundancy Detection Process

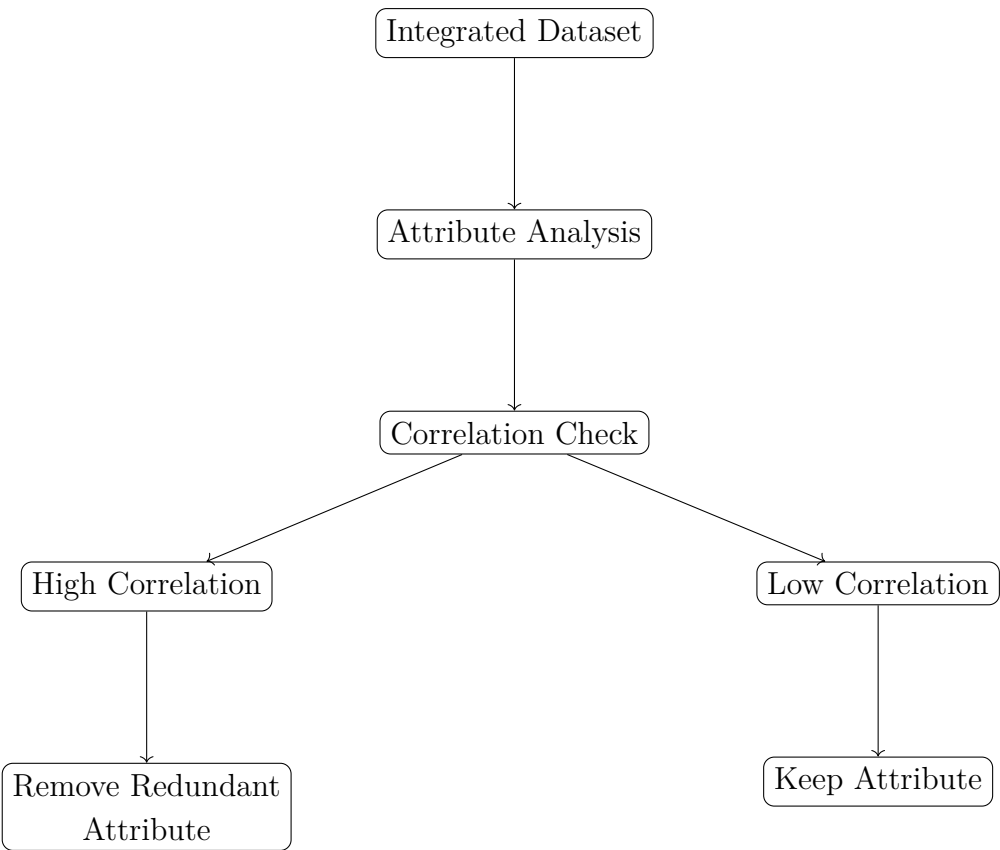


Figure 2.8: Redundancy Detection Process

Advantages of Redundancy Removal

- Reduces storage requirements.

- Improves query performance.
- Simplifies data analysis.
- Improves data mining efficiency.

2.4 Data Transformation

Data Transformation is the process of converting data into suitable formats for data mining and machine learning. It improves data quality, consistency, and mining efficiency.

2.4.1 Objectives of Data Transformation

- Improve data quality.
- Standardize attribute values.
- Reduce complexity.
- Create meaningful features.
- Improve mining accuracy.

2.4.2 Normalization

Normalization scales attribute values into a specified range so that attributes with large values do not dominate attributes with small values.

Need for Normalization

- Improves machine learning performance.
- Eliminates scale differences.
- Speeds up convergence of algorithms.

Min-Max Normalization

Min-Max normalization transforms data into the range $[0,1]$.

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

where:

- x = original value
- x_{min} = minimum value

- x_{max} = maximum value

Example:

$$x = 50, \quad x_{min} = 20, \quad x_{max} = 100$$

$$x' = \frac{50 - 20}{100 - 20} = \frac{30}{80} = 0.375$$

Z-Score Normalization

Z-score normalization uses mean and standard deviation.

$$z = \frac{x - \mu}{\sigma}$$

where:

- μ = mean
- σ = standard deviation

Decimal Scaling

$$x' = \frac{x}{10^j}$$

where j is chosen such that:

$$|x'| < 1$$

Normalization Process

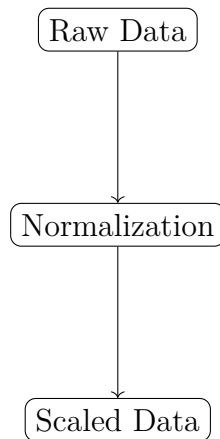


Figure 2.9: Normalization Process

2.4.3 Attribute Construction

Attribute Construction generates new attributes from existing attributes to improve the quality of analysis.

Example

Original Attributes:

Date of Birth	Current Year
2000	2025

Table 2.8: Original Attributes

Constructed Attribute:

$$Age = Current\ Year - Birth\ Year$$

$$Age = 2025 - 2000 = 25$$

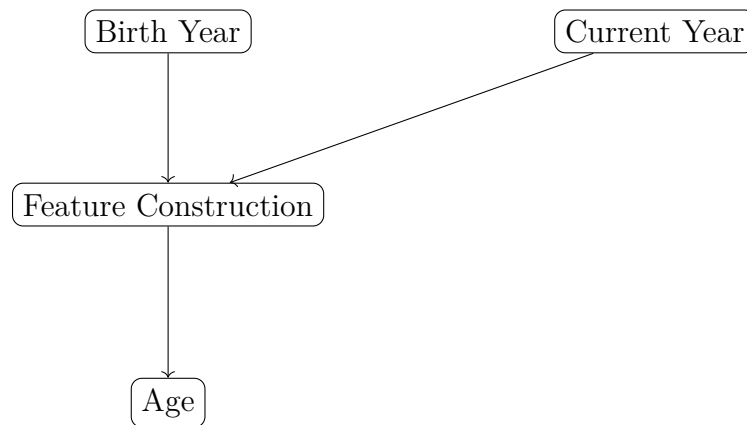
Attribute Construction Diagram

Figure 2.10: Attribute Construction Process

2.4.4 Discretization

Discretization converts continuous attributes into finite intervals or categories.

Example

Age Values:

18, 22, 35, 48, 60

After discretization:

Age Range	Category
0-18	Child
19-35	Young
36-55	Adult
56+	Senior

Table 2.9: Discretization Example

Equal Width Binning

Bin Width:

$$W = \frac{Max - Min}{k}$$

where:

- Max = Maximum value
- Min = Minimum value
- k = Number of bins

Example:

$$Max = 100, \quad Min = 0, \quad k = 5$$

$$W = \frac{100 - 0}{5} = 20$$

Bins:

$$[0, 20), [20, 40), [40, 60), [60, 80), [80, 100]$$

Discretization Process



Figure 2.11: Equal Width Binning

2.4.5 Concept Hierarchy Generation

Concept Hierarchy Generation replaces low-level concepts with higher-level concepts to simplify analysis.

Example

Location hierarchy:

$$Kankinara \rightarrow North\ 24\ Parganas \rightarrow West\ Bengal \rightarrow India$$

Advantages

- Reduces data complexity.
- Supports data summarization.
- Improves OLAP analysis.

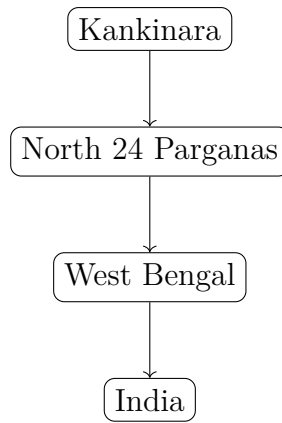
Concept Hierarchy Diagram

Figure 2.12: Concept Hierarchy Generation

2.4.6 Advantages of Data Transformation

- Improves data quality.
- Reduces redundancy.
- Enhances mining efficiency.
- Improves model accuracy.
- Simplifies data analysis.

2.5 Data Reduction

Data Reduction is the process of reducing the volume of data while maintaining the integrity and analytical quality of the original dataset. It helps reduce storage requirements, computational costs, and processing time while preserving meaningful information.

2.5.1 Objectives of Data Reduction

- Reduce storage space.
- Improve mining efficiency.
- Reduce computational complexity.
- Eliminate irrelevant attributes.
- Speed up data analysis.

2.5.2 Data Cube Aggregation

Data Cube Aggregation summarizes detailed data into higher-level aggregated forms.

Example

Daily Sales Data:

Day	Sales
Monday	1000
Tuesday	1200
Wednesday	1100

Table 2.10: Daily Sales Data

Aggregated Monthly Sales:

$$Total\ Sales = \sum_{i=1}^n Sales_i$$

$$Total = 1000 + 1200 + 1100 = 3300$$

Aggregation Process

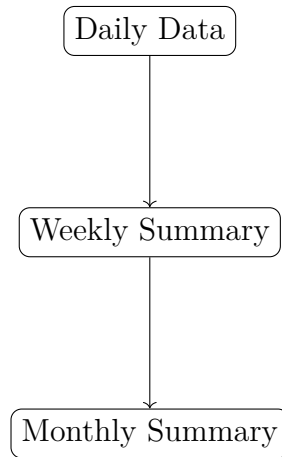


Figure 2.13: Data Cube Aggregation

2.5.3 Attribute Subset Selection

Attribute Subset Selection identifies and removes irrelevant or redundant attributes while retaining useful information.

Example

Original Attributes:

- Student ID
- Name
- Age
- Phone Number
- Exam Score

Selected Attributes:

- Age
- Exam Score

Benefits

- Reduces overfitting.
- Improves model performance.
- Simplifies analysis.

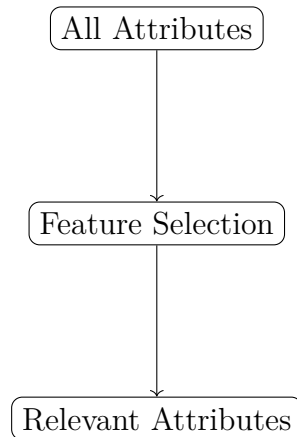
Attribute Selection Process

Figure 2.14: Attribute Subset Selection

2.5.4 Numerosity Reduction

Numerosity Reduction replaces large datasets with smaller representations while preserving essential information.

Techniques

- Sampling
- Regression Models
- Clustering
- Histograms

Sampling Formula

For Simple Random Sampling:

$$n = \frac{N}{1 + Ne^2}$$

where:

- n = Sample size
- N = Population size
- e = Margin of error

Example:

$$N = 1000, \quad e = 0.05$$

$$n = \frac{1000}{1 + 1000(0.05)^2}$$

$$n \approx 286$$

Sampling Diagram

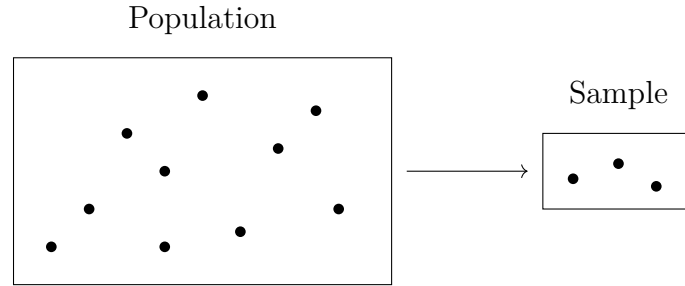


Figure 2.15: Sampling for Numerosity Reduction

2.5.5 Dimensionality Reduction

Dimensionality Reduction reduces the number of attributes while retaining important information.

Benefits

- Faster computation.
- Reduced storage requirements.
- Improved visualization.
- Reduced overfitting.

Principal Component Analysis (PCA)

PCA transforms original variables into a smaller set of principal components.

Covariance Matrix:

$$Cov(X, Y) = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{n - 1}$$

Principal Components are obtained from:

$$Av = \lambda v$$

where:

- A = Covariance matrix

- λ = Eigenvalue
- v = Eigenvector

PCA Reduction Process

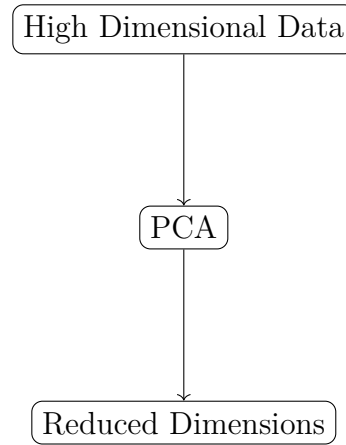


Figure 2.16: Dimensionality Reduction using PCA

2.5.6 Data Compression

Data Compression reduces the amount of storage space required by data.

Types of Compression

1. Lossless Compression
2. Lossy Compression

Compression Ratio

The compression ratio is:

$$\text{Compression Ratio} = \frac{\text{Original Size}}{\text{Compressed Size}}$$

Example:

$$\text{Compression Ratio} = \frac{100 \text{ MB}}{25 \text{ MB}} = 4$$

This indicates a 4:1 compression ratio.

Compression Workflow

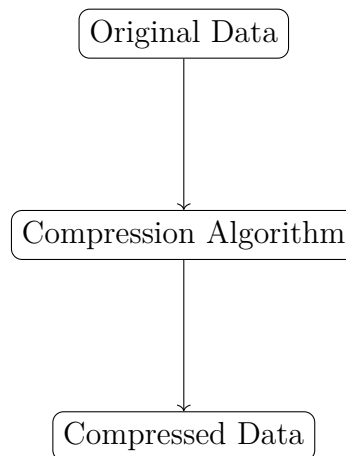


Figure 2.17: Data Compression Process

2.5.7 Advantages of Data Reduction

- Reduces storage requirements.
- Speeds up data mining algorithms.
- Reduces computational costs.
- Improves scalability.
- Facilitates efficient data analysis.

2.6 Data Wrangling

Data Wrangling, also known as Data Munging, is the process of cleaning, transforming, and organizing raw data into a suitable format for analysis and machine learning. Real-world datasets often contain missing values, duplicate records, inconsistent formats, and noisy data. Data wrangling helps improve data quality and reliability.

Major Tasks in Data Wrangling

- Data Collection
- Data Cleaning
- Handling Missing Values
- Removing Duplicates
- Data Transformation

- Data Integration
- Data Validation

Example

Consider the dataset:

Name	Age	Salary
John	25	50000
Alice	-	60000
John	25	50000
Bob	30	-

Table 2.11: Raw Dataset

After data wrangling:

- Missing values are filled.
- Duplicate rows are removed.
- Data types are standardized.

Python Libraries

- Pandas
- NumPy
- OpenPyXL
- PyJanitor

Data wrangling improves data quality and ensures accurate analysis and model performance.

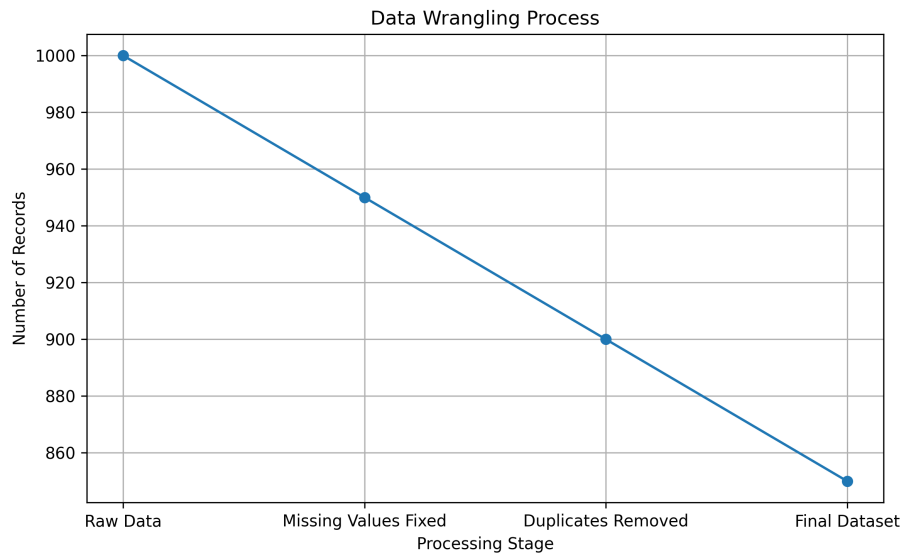


Figure 2.18: Data Wrangling Process

2.7 Data Visualization

Data Visualization is the graphical representation of data and information using charts, graphs, and plots. It helps identify patterns, trends, outliers, and relationships within datasets.

Visualization makes complex datasets easier to understand and supports better decision-making.

Common Types of Visualization

- Bar Chart
- Line Chart
- Scatter Plot
- Histogram
- Pie Chart
- Box Plot
- Heatmap

Python Libraries

- Matplotlib
- Seaborn

- Plotly
- Bokeh
- Altair

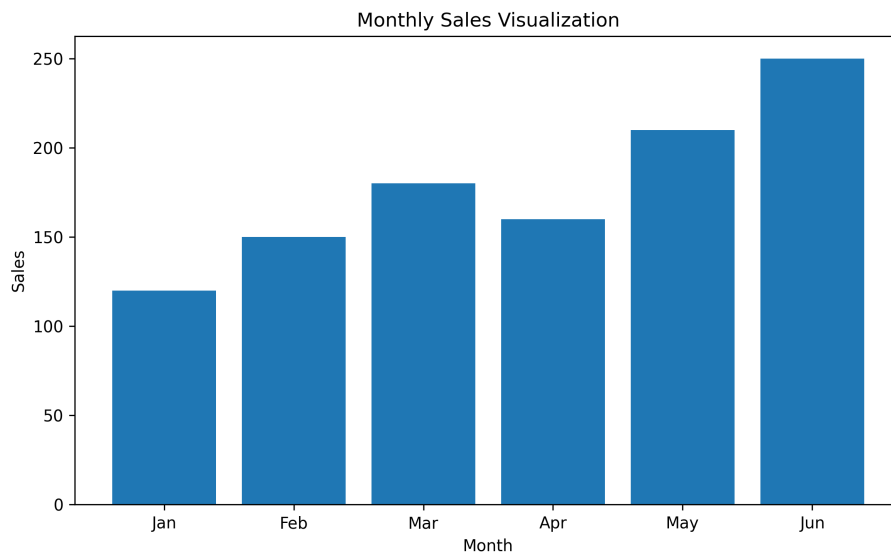


Figure 2.19: Monthly Sales Visualization

2.7.1 Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) is the process of examining and summarizing datasets before applying machine learning algorithms. EDA helps understand data characteristics, detect anomalies, identify relationships between variables, and generate hypotheses.

Objectives of EDA

- Understand data distribution
- Detect outliers
- Identify missing values
- Find correlations
- Discover patterns and trends

Common EDA Techniques

- Descriptive Statistics
- Histograms

- Scatter Plots
- Box Plots
- Correlation Analysis
- Pair Plots

Python Libraries Used

- Pandas
- NumPy
- Matplotlib
- Seaborn
- Plotly
- Sweetviz
- ydata-profiling

EDA is an essential step in the data science workflow because it helps analysts understand the structure and quality of data before model development.

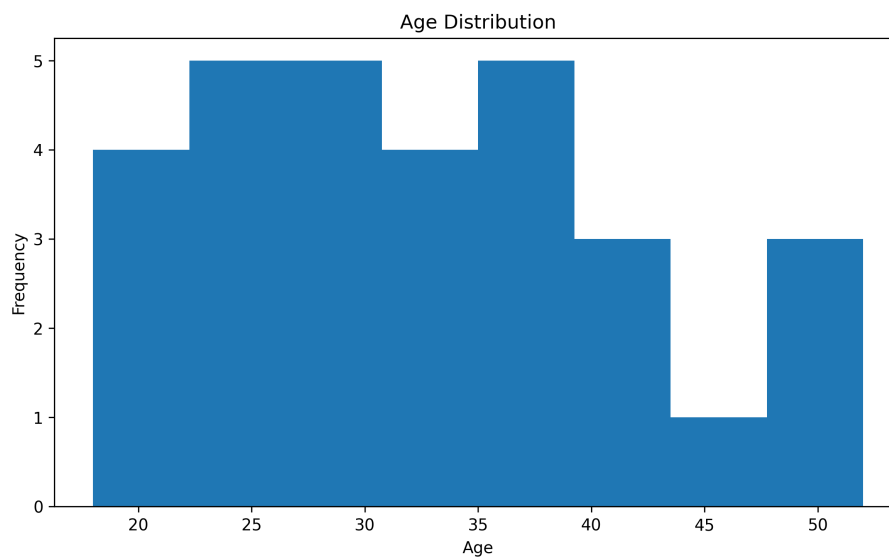


Figure 2.20: Histogram used in Exploratory Data Analysis

Chapter 3

Supervised Learning

Supervised Learning is a type of machine learning where the model is trained using labeled data. The algorithm learns a mapping function between input (X) and output (Y).

$$Y = f(X) \quad (3.1)$$

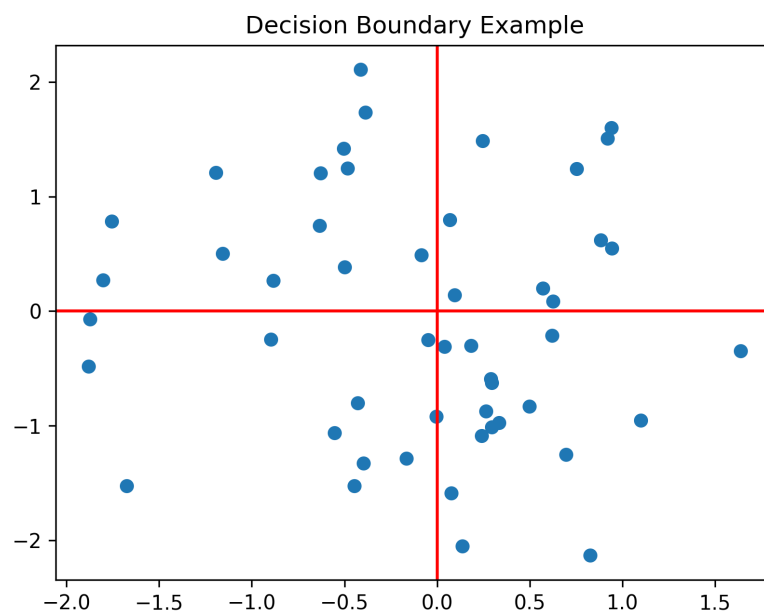


Figure 3.1: Decision Boundary Representation

It is mainly divided into:

- Classification (discrete output)
- Regression (continuous output)

3.1 Classification

The key objective of classification-based tasks is to predict categorical output labels or responses for the given input data such as true-false, male-female, yes-no etc. As we know, the categorical output responses mean unordered and discrete values; hence, each output response will belong to a specific class or category.

Some popular classification algorithms are Logistic Regression, Decision Tree Classifier, Random Forest Classifier, Support Vector Machine(SVM), Naive Bayes, KNN.

3.1.1 Use-Cases:

- Email: Spam/Not Spam
- Student Result: pass/fail
- Disease Detection: positive/negative

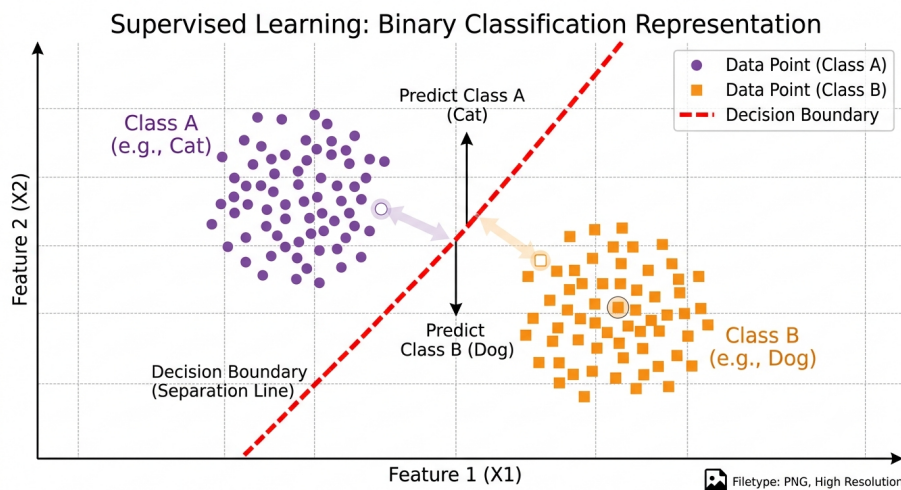


Figure 3.2: Classification Representation

3.2 Regression

The key objective of regression-based tasks is to predict output labels or responses, which are continuous numeric values, for the given input data. Basically, regression models use the input data features (independent variables) and their corresponding continuous numeric output values (dependent or outcome variables) to learn specific associations between inputs and corresponding outputs.

Some popular regression algorithms are linear regression, Multiple Linear Regression, polynomial regression, Lasso regression, Decision Tree Regression, Support Vector Regression(SVR).

3.2.1 Use-Cases:

- Predict House Price
- Predict salary based on experiences
- All labeled numerical data used in regression

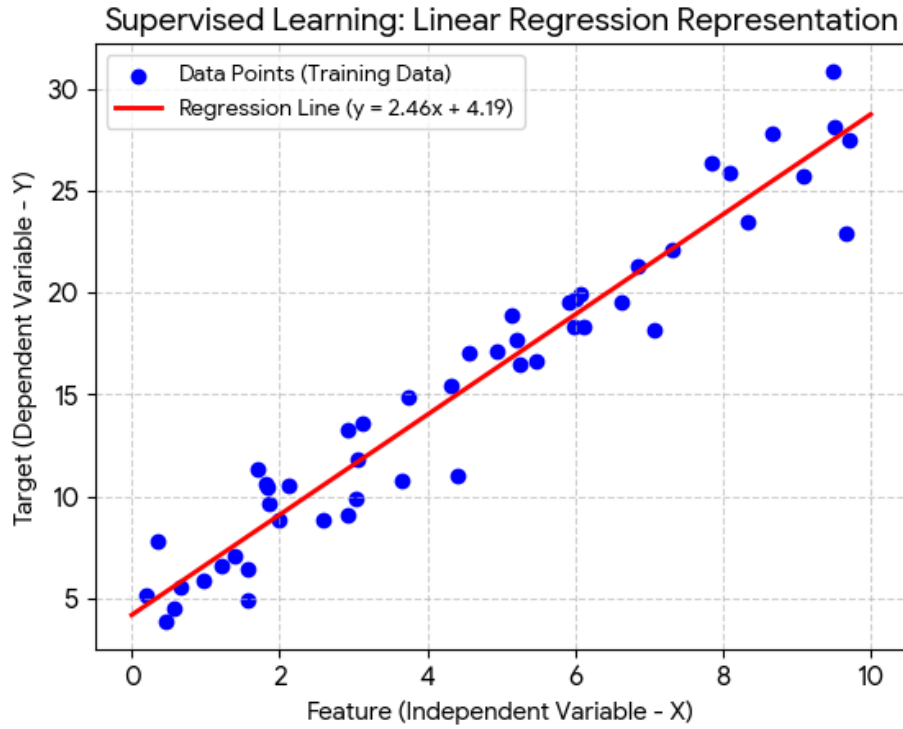


Figure 3.3: Regression Representation

3.3 Supervised Learning Algorithms

3.3.1 Linear Regression:

It is a method used to describe how a target variable (dependent variable Y) changes with one or more input features (Independent variable X) by fitting a straight line relationship between them.

The model assumes that the target value is:

$$Y = \beta_0 + \beta_1 X + \epsilon \quad (3.2)$$

Where:

- Y = Target Variable
- X = Input Feature

- β_0 = Y-intercept (Mean value of Y when $X = 0$)
- β_1 = Slope or Coefficient (Change in Y when X changes by one unit)
- ϵ = Error Term (Difference between observed value of Y and predicted value of the linear model)

It is called Linear because:

- The relationship between input features and output is represented as a straight line.
- The model does not involve non-linear interactions between variables.
- **Linearity in parameters:** The equation $Y = \beta_0 + \beta_1 X$ defines the target as a linear combination of parameters.

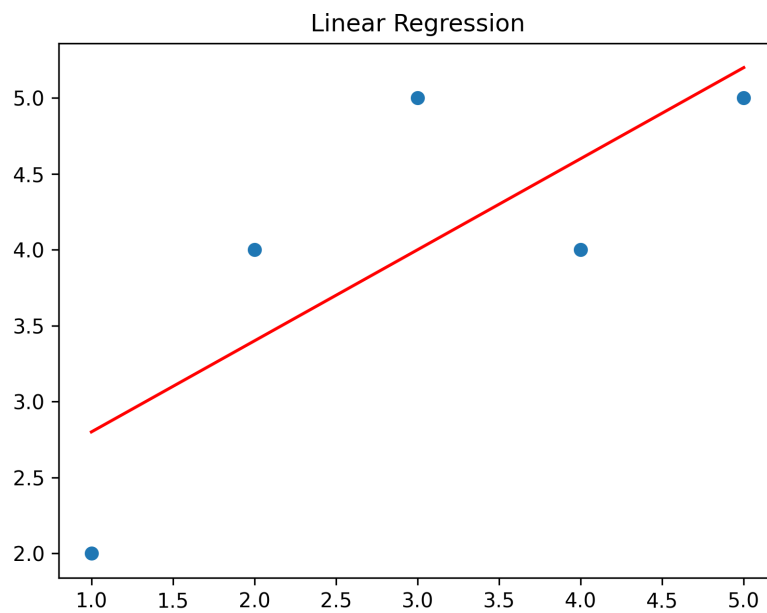


Figure 3.4: Linear Regression

3.3.2 K-Nearest Neighbors:

K-Nearest Neighbors (KNN) is a supervised learning algorithm used for classification and regression. It classifies a new data point based on the majority label of its nearest neighbors.

Key Idea:

- No training phase (lazy learning)

- Uses distance metrics
- Prediction depends on K nearest points

Distance Formulas

Distance plays a major role in KNN.

i. Euclidean Distance

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} \quad (3.3)$$

ii. Manhattan Distance

$$d(x, y) = \sum_{i=1}^n |x_i - y_i| \quad (3.4)$$

iii. Minkowski Distance

$$d(x, y) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{\frac{1}{p}} \quad (3.5)$$

Special Cases:

- $p = 1 \rightarrow$ Manhattan Distance
- $p = 2 \rightarrow$ Euclidean Distance

Choosing the Value of K

The value of K is very important:

- Small K (e.g., K=1): Captures fine details but makes the model highly sensitive to noise, outliers, and errors, leading to overfitting.
- Large K: Smooths out boundaries but risks capturing data points from completely unrelated classes, leading to underfitting.

Best Practice:

- Use cross-validation
- Choose odd K for binary classification
- Common rule: $K = \sqrt{N}$

Graph Representation of KNN

KNN can be visualized in a 2D feature space.

- Each point represents a data sample
- Distance is calculated from the new point
- K nearest points are selected

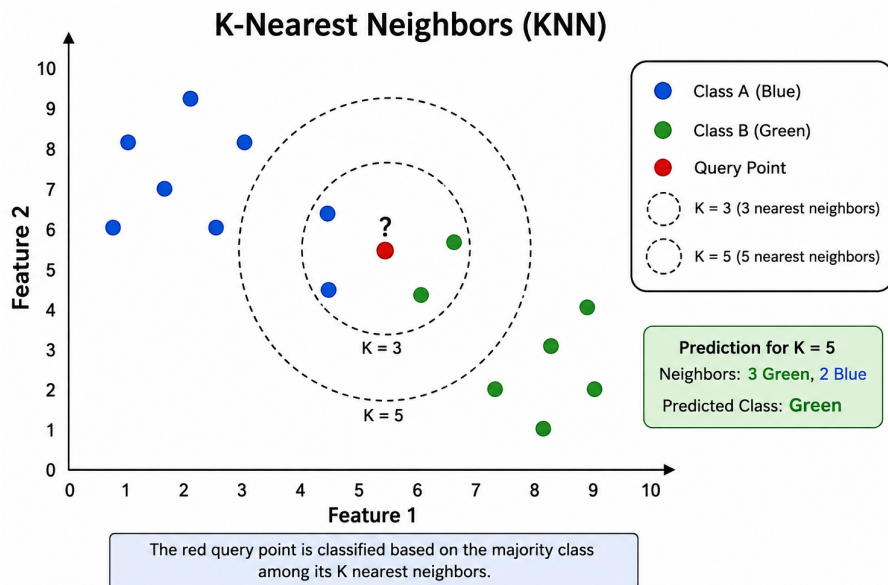


Figure 3.5: KNN Representation

Explanation of Graph:

- Red point = Query point
- Blue/Green points = Training data
- Circle = Region of K nearest neighbors

KNN Algorithm Steps

1. Choose K value
2. Compute distance between test point and all training points
3. Select K nearest neighbors
4. Perform majority voting (classification)
5. Return predicted class

3.3.3 Decision Trees:

A decision tree is a non-parametric supervised machine learning algorithm that uses a flowchart-like hierarchical structure to classify data or predict continuous numerical outcomes. It breaks down a complex dataset into smaller, more homogeneous subsets by asking sequential, condition-based questions.

Graph Representation

A decision tree is represented as a hierarchical structure:

- Root Node: Entire dataset
- Internal Nodes: Feature-based decision rules
- Leaf Nodes: Final output (class/label)

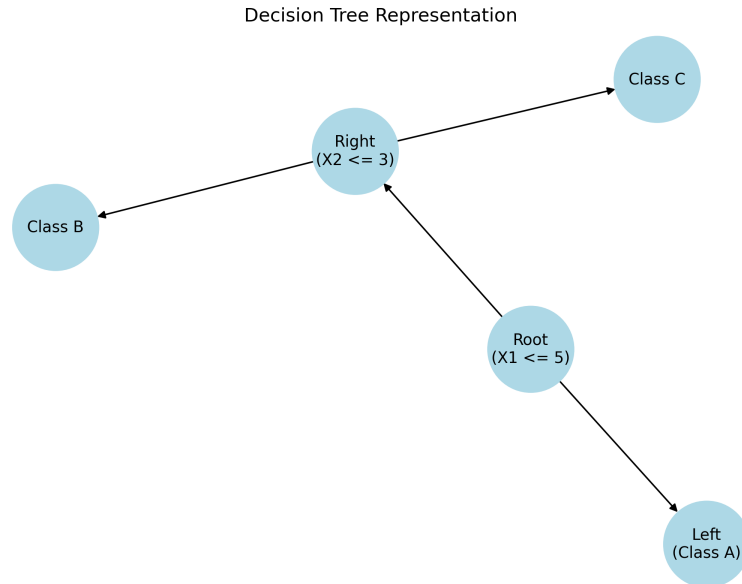


Figure 3.6: Decision Tree Representation

Splitting Criteria

1. Entropy:

In information theory, entropy quantifies how uncertain a dataset is.

$$H(S) = - \sum_{i=1}^c p_i \log_2(p_i)$$

2. Information Gain:

It tells us how much entropy (uncertainty) decreases after a split. It also known

as "Reduction in Entropy".

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

3. Gini Index:

It is alternative impurity measure, used for measure probability of misclassifications. Typically used in CART. Lower Gini = more pure node.

$$\text{Gini}(S) = 1 - \sum_{i=1}^c p_i^2$$

Threshold Values

Entropy Scores:

- **Near 0 (Perfect Split / High Purity):** This means the group contains data points that are mostly or entirely of the same class. The data is highly organized, making it a great split.
- **Near 1 (Poor Split / High Impurity):** This means the group contains a 50/50 mix of different classes. The data is completely random, making it a bad split because it failed to separate the categories.

Information Gain:

- **Near to 1 (High Information Gain):** This is a good split. The data goes from being mixed (chaotic) to being cleanly separated into distinct, pure groups. You successfully "gained a lot of information" to make a decision.
- **Near to 0 (Low/No Information Gain):** This is a bad split. The split leaves the child nodes just as mixed up and random as the parent node was. You gained zero new information to help classify the data.

3.3.4 Naive Bayes:

Naive Bayes is a collection of supervised machine learning classification algorithms based on Bayes' Theorem. It calculates the posterior probability of a class given a set of features and assigns the data point to the class with the highest probability. The algorithm is fundamentally defined by its "naive" independence assumption, which presumes that all input features are completely independent of one another given the target class variable.

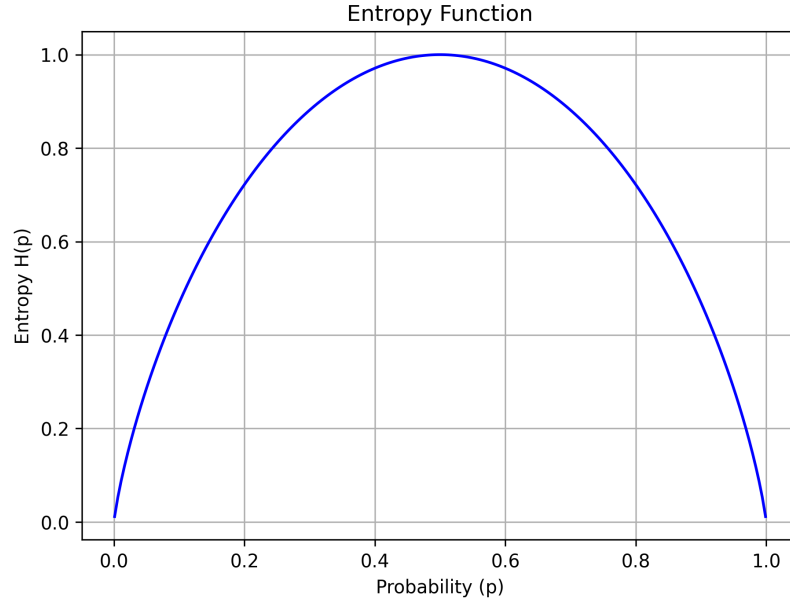


Figure 3.7: Entropy vs Probability

Bayes Theorem

$$P(C|X) = \frac{P(X|C) P(C)}{P(X)}$$

Where:

- $P(C|X)$ = Posterior probability
- $P(X|C)$ = Likelihood
- $P(C)$ = Prior probability
- $P(X)$ = Evidence

Naive Assumption

$$P(X|C) = \prod_{i=1}^n P(x_i|C)$$

Thus:

$$P(C|X) = \frac{P(C) \prod_{i=1}^n P(x_i|C)}{P(X)}$$

Example Visualization**Bayesian Network**

A Bayesian Network is a graphical model representing probabilistic relationships using a Directed Acyclic Graph (DAG). A Bayesian Network represents:

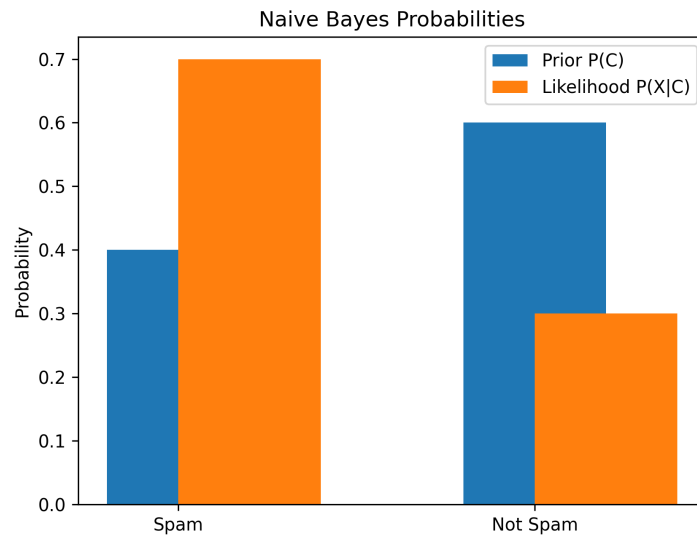


Figure 3.8: Naive Bayes Probability

- Nodes = Random variables
- Edges = Conditional dependencies

Joint Probability Representation

$$P(X_1, X_2, \dots, X_n) = \prod_{i=1}^n P(X_i | \text{Parents}(X_i))$$

Graph Representation

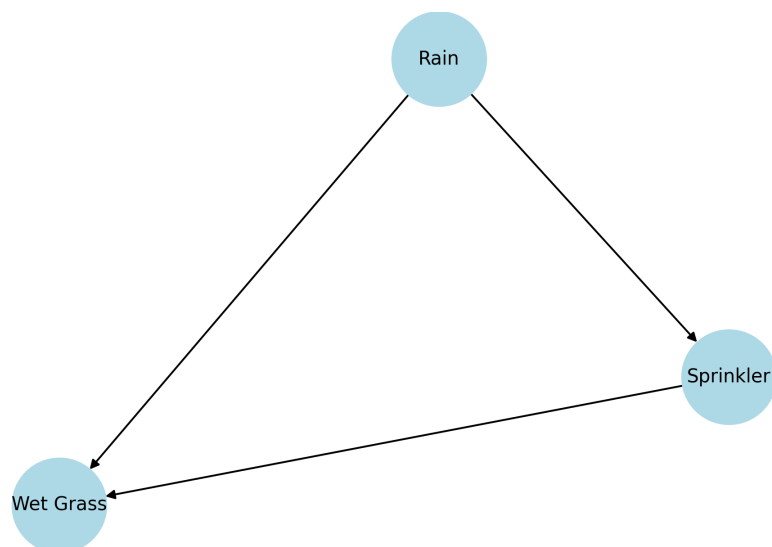


Figure 3.10: Bayesian Network

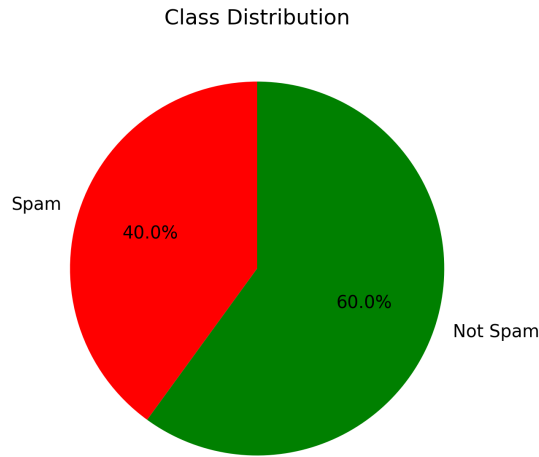


Figure 3.9: Class Distribution

3.3.5 Logistic Regression:

Logistic Regression is a supervised learning algorithm used for binary classification tasks in ML. Here the sigmoid function is used for transforms the linear combinations of input into probability between 0 and 1, Which maintain the valid range.

Sigmoid Function

The sigmoid function is defined as:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Properties

- $\sigma(z) \rightarrow 1$ as $z \rightarrow +\infty$
- $\sigma(z) \rightarrow 0$ as $z \rightarrow -\infty$
- $\sigma(z) \in (0, 1)$

Hypothesis Function

$$h_{\theta}(x) = \sigma(w^T x + b)$$

It gives probability:

$$P(y = 1|x)$$

Decision Rule

$$\hat{y} = \begin{cases} 1 & \text{if } h_{\theta}(x) \geq 0.5 \\ 0 & \text{otherwise} \end{cases}$$

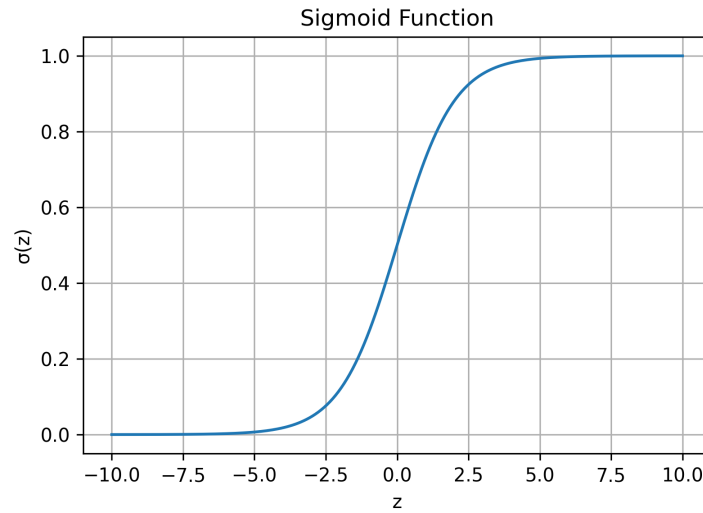
Sigmoid Graph Representation

Figure 3.11: Sigmoid Function Curve

3.3.6 Support Vector Machines(SVM):

A support vector machine finds the best hyperplane that separates data points of different classes with the maximum margin. The hyperplane defined as $W.X+b = 0$ [where W = weight vector, X = Input features, b = bias].

Components of SVM**1. Hyperplane**

A hyperplane is a decision boundary that separates different classes.

2. Support Vectors

Support vectors are the closest data points to the hyperplane that influence its position.

3. Margin

Margin is the distance between the hyperplane and the nearest support vectors. SVM tries to maximize this margin.

$$\text{Margin} = \frac{2}{\|w\|}$$

4. Weight Vector (w)

It defines the orientation of the hyperplane.

5. Bias (b)

It shifts the hyperplane to properly separate classes.

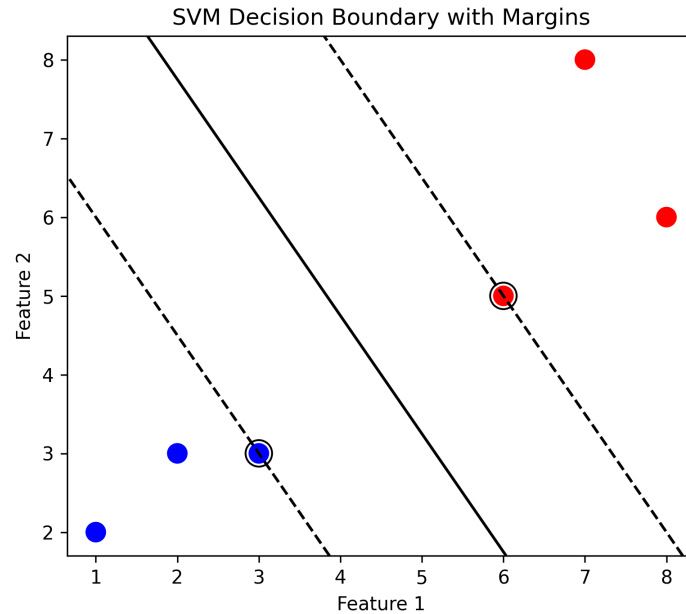


Figure 3.12: SVM Decision Boundary with Support Vectors

Objective of SVM

$$\min \frac{1}{2} \|w\|^2$$

subject to:

$$y_i(w^T x_i + b) \geq 1$$

Kernel Trick

When data is not linearly separable, SVM uses the Kernel Trick to transform data into higher dimensions.

Idea

Instead of explicitly mapping data to higher dimensions, SVM uses a kernel function:

$$K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$$

where $\phi(x)$ is a transformation function.

Types of Kernel Functions

- Linear Kernel: $K(x_i, x_j) = x_i^T x_j$
- Polynomial Kernel: $K(x_i, x_j) = (x_i^T x_j + c)^d$
- Radial Basis Function (RBF): $K(x_i, x_j) = e^{-\gamma \|x_i - x_j\|^2}$
- Sigmoid Kernel: $K(x_i, x_j) = \tanh(\alpha x_i^T x_j + c)$

3.3.7 Random Forest:

Random Forest is a supervised machine learning algorithm that belongs to the ensemble learning method. It is used for both classification and regression tasks. It builds multiple decision trees and combines their outputs to improve accuracy and reduce overfitting.

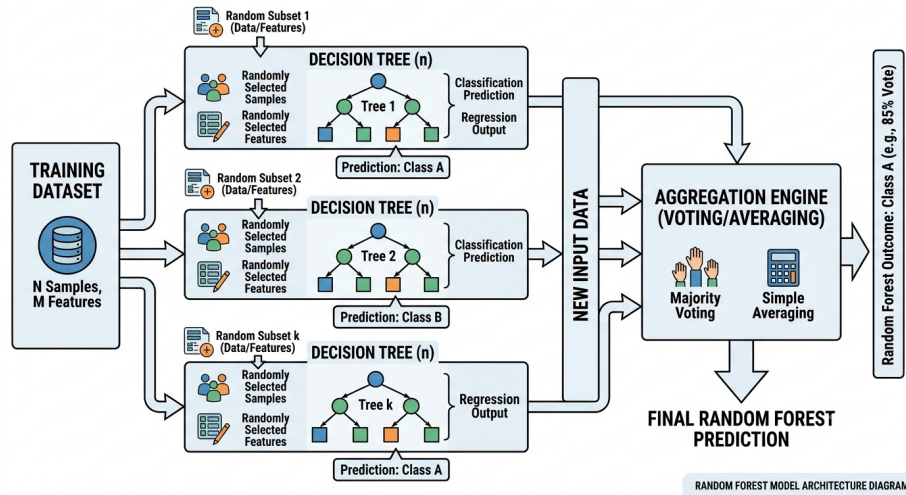


Figure 3.13: Random Forest Architecture

Basic Idea

Random Forest works by creating multiple decision trees on different subsets of the dataset and then combining their predictions using:

- Majority voting (for classification)
- Average (for regression)

Working of Random Forest

1. Select random samples from the dataset (Bootstrap sampling)
2. Build a decision tree for each sample
3. At each split, select random features

4. Repeat for many trees
5. Combine all predictions

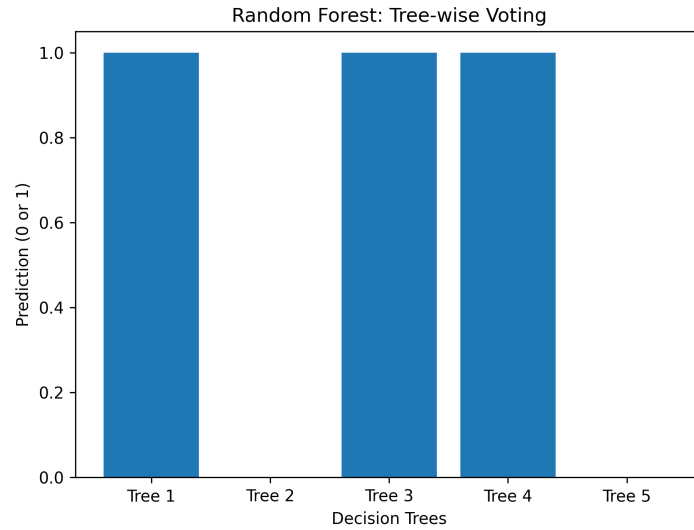


Figure 3.14: Random Forest Voting Representation

Bootstrap Aggregation (Bagging)

Random Forest uses Bagging:

Bagging = Bootstrap samples + Aggregation

It reduces variance and improves model stability.

Prediction

Classification

$\hat{y} = \text{mode of all tree predictions}$

Regression

$$\hat{y} = \frac{1}{N} \sum_{i=1}^N y_i$$

Important Parameters

- Number of trees (n_estimators)
- Maximum depth of tree
- Number of features considered at each split

Chapter 4

Ensemble Modeling

Ensemble Learning is a machine learning technique where multiple base models are combined and provide a string predictive model to improve performance, accuracy, and stability compared to a single model. Main types:

- Bagging
- Boosting
- Stacking

4.1 Bagging (Bootstrap Aggregation)

4.1.1 Algorithm

1. Create multiple bootstrap samples from dataset
2. Train separate models (usually decision trees)
3. Combine predictions using voting (classification) or averaging (regression)

4.1.2 Mathematical Idea

$$\hat{y} = \frac{1}{N} \sum_{i=1}^N h_i(x)$$

4.1.3 Example

Random Forest is the best example of Bagging.

4.1.4 Architecture

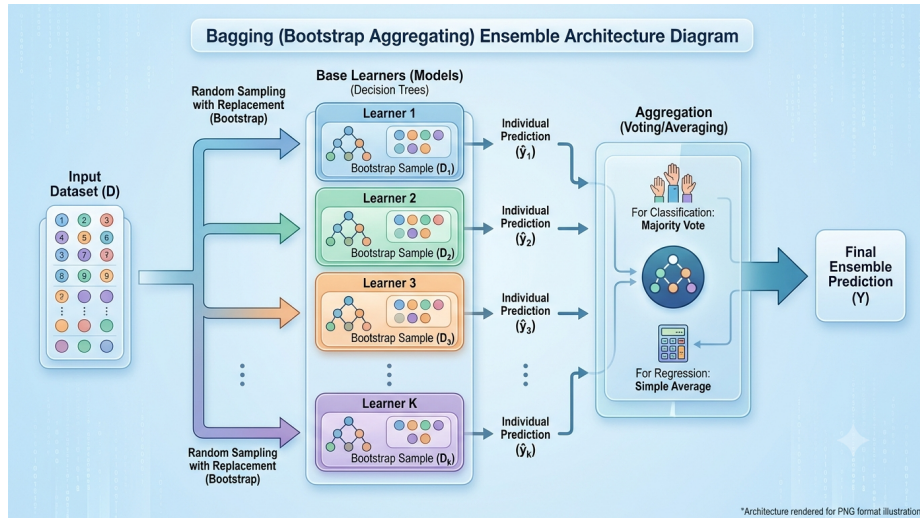


Figure 4.1: Bagging Architecture

4.2 Boosting

4.2.1 Algorithm

1. Train weak model on dataset
2. Increase weight of misclassified samples
3. Train next model focusing on errors
4. Combine all models sequentially

4.2.2 Mathematical Idea

$$F(x) = \sum_{i=1}^N \alpha_i h_i(x)$$

4.2.3 Example

AdaBoost, Gradient Boosting, XGBoost

4.2.4 Architecture

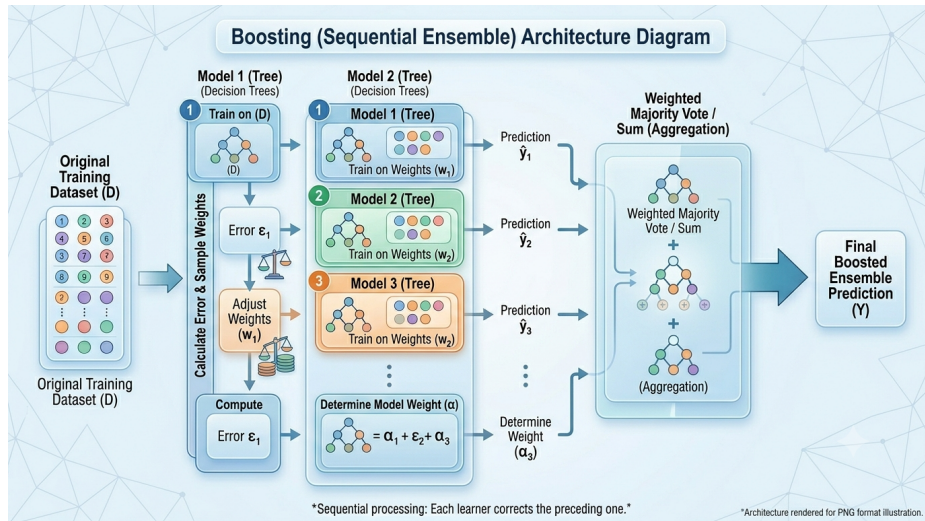


Figure 4.2: Boosting Architecture

4.3 Stacking

4.3.1 Algorithm

1. Train multiple base models
2. Collect predictions from base models
3. Use meta-model to learn final prediction

4.3.2 Architecture

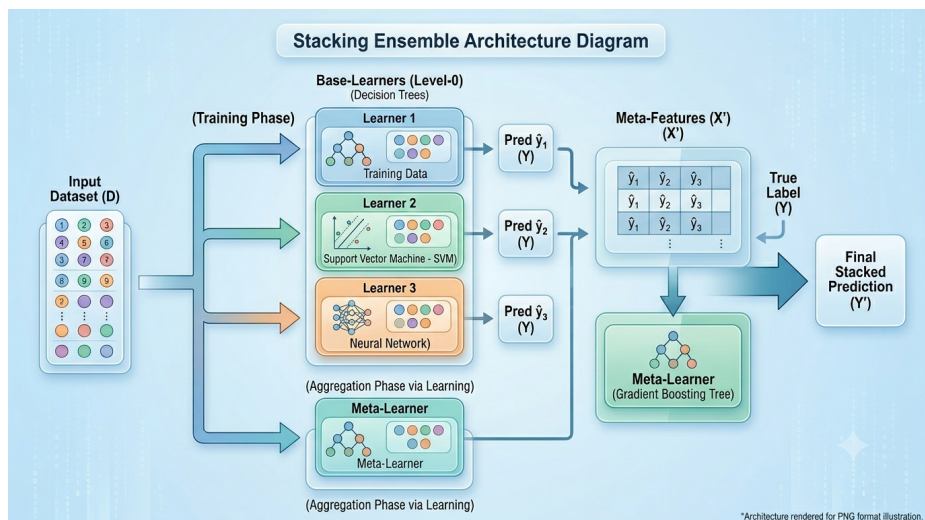


Figure 4.3: Stacking Architecture

Chapter 5

Unsupervised Learning

Unsupervised learning is a class of machine learning where an algorithm learns patterns from unlabeled data — data that has no pre-assigned target outputs (y). The goal is to discover hidden structure, distributions, or representations in the input space X .

Formal Definition: Given a dataset $X = \{x_1, x_2, \dots, x_n\}$ where $x_i \in \mathbb{R}^d$, find a function $f(X)$ that captures the underlying structure without using any label information.

5.1 Key Terminologies

Term	Definition
Feature Space	High-dimensional space $\mathbb{R}^d$ where each sample is a point
Latent Space	Lower-dimensional representation learned by the model
Cluster	A group of data points that are more similar to each other than to others
Density	Number of data points per unit volume in feature space
Manifold	A lower-dimensional surface embedded in high-dimensional space
Reconstruction	Re-creating input from compressed latent representation

Table 5.1: Key Terminologies

5.2 Clustering Algorithms

5.2.1 K-Means Clustering

K-Means is one of the most popular unsupervised machine learning algorithms used for partitioning a dataset into K distinct clusters. The algorithm groups similar data points together while maximizing the separation between different clusters.

Mathematical Representation

Given a dataset

$$X = \{x_1, x_2, \dots, x_n\}, \quad x_i \in \mathbb{R}^d$$

the objective of K-Means is to partition the data into K clusters

$$C = \{C_1, C_2, \dots, C_K\}$$

such that the within-cluster variance is minimized.

Cluster Centroid The centroid of cluster C_k is calculated as:

$$\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$$

where:

- μ_k = centroid of cluster k
- $|C_k|$ = number of samples in cluster k

Loss Function (Objective Function) K-Means minimizes the Within-Cluster Sum of Squares (WCSS):

$$J = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

where:

- J = clustering loss
- $\|x_i - \mu_k\|^2$ = squared Euclidean distance

The optimal clustering minimizes J .

Distance Metric Most commonly, Euclidean distance is used:

$$d(x_i, \mu_k) = \sqrt{\sum_{j=1}^d (x_{ij} - \mu_{kj})^2}$$

Working Algorithm

1. Choose the number of clusters K .
2. Initialize K centroids randomly.
3. Assign each data point to the nearest centroid.
4. Recalculate cluster centroids.
5. Repeat steps 3 and 4 until convergence.

Graph Representation

The figure below illustrates the iterative process of K-Means clustering.

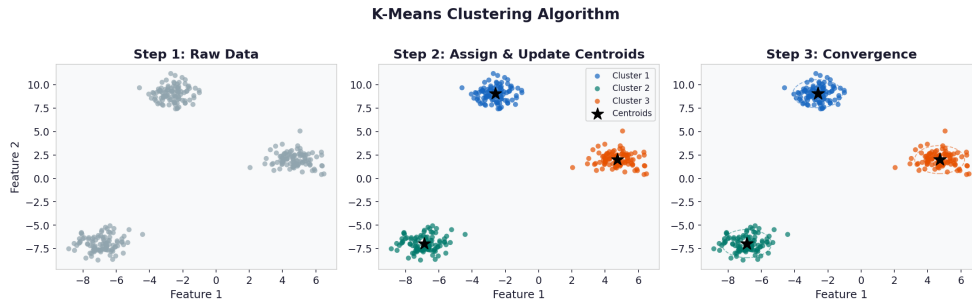


Figure 5.1: K-Means: raw data $\rightarrow$ cluster assignment $\rightarrow$ convergence

K-Means++

K-Means++ is an improved centroid initialization technique that reduces the probability of poor clustering caused by random initialization.

Procedure

1. Select the first centroid randomly.
2. Compute the distance of each point from the nearest selected centroid.
3. Select the next centroid with probability proportional to the squared distance.
4. Repeat until K centroids are selected.
5. Run standard K-Means.

Selection Probability The probability of selecting a point x as a centroid is

$$P(x) = \frac{D(x)^2}{\sum_{x_i \in X} D(x_i)^2}$$

where $D(x)$ is the distance from the nearest chosen centroid.

Advantages

- Faster convergence.
- Better cluster quality.
- Reduces sensitivity to random initialization.

K Assignment Methods

Selecting an appropriate value of K is crucial for obtaining meaningful clusters.

i. Elbow Method

The Elbow Method evaluates clustering performance for different values of K .

For each K , compute:

$$WCSS(K) = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

The WCSS decreases as K increases. The optimal value of K is chosen at the "elbow point," where the rate of decrease sharply changes.

Interpretation

- Small K : Underfitting.
- Large K : Overfitting.
- Elbow Point: Best trade-off between complexity and performance.

Example elbow curve:

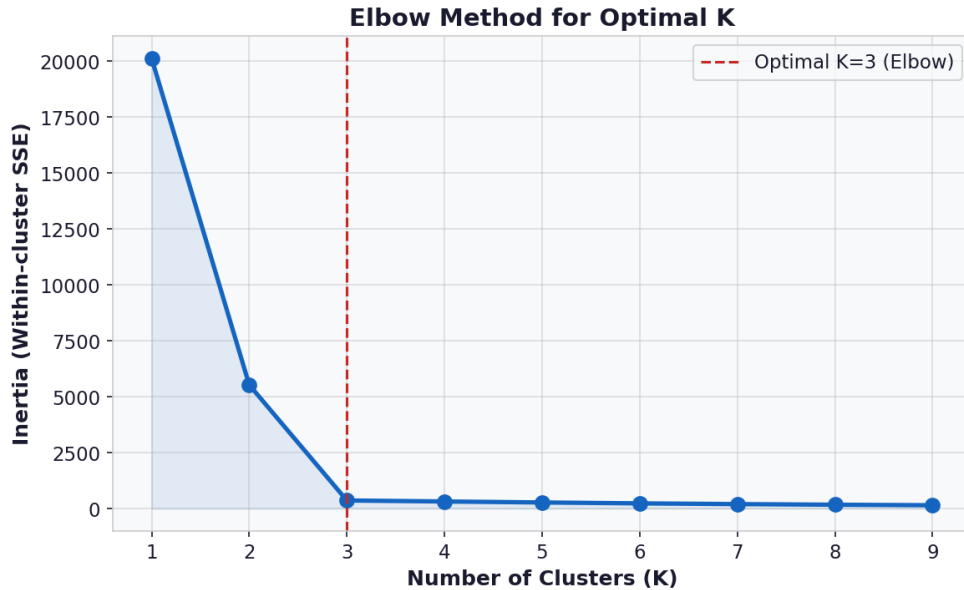


Figure 5.2: Elbow Method for choosing optimal K (inflection at K=3)

ii. Silhouette Method

The Silhouette Score measures how similar a data point is to its own cluster compared to other clusters.

For a sample i ,

$a(i)$ = average distance to points within the same cluster

$b(i)$ = minimum average distance to points in another cluster

The silhouette score is:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

where

$$-1 \leq s(i) \leq 1$$

Interpretation

- $s(i) \approx 1$: Well clustered.
- $s(i) \approx 0$: On cluster boundary.
- $s(i) < 0$: Possibly assigned to the wrong cluster.

The optimal value of K is the one that produces the highest average silhouette score.

Advantages of Silhouette Method

- Considers both cohesion and separation.
- More reliable than Elbow Method when the elbow is unclear.
- Provides a quantitative measure of clustering quality.

Advantages of K-Means

- Simple and computationally efficient.
- Scales well to large datasets.
- Easy to implement and interpret.

Limitations of K-Means

- Requires specifying K beforehand.
- Sensitive to initialization.
- Assumes spherical clusters.
- Affected by outliers and noise.

5.2.2 DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

DBSCAN is a density-based unsupervised clustering algorithm that groups together data points that are closely packed while identifying points in low-density regions as outliers (noise). Unlike K-Means, DBSCAN does not require the number of clusters to be specified beforehand and can discover clusters of arbitrary shapes.

A cluster C satisfies:

$$\forall p, q \in C, \quad p \text{ and } q \text{ are density connected}$$

Parameters

DBSCAN primarily depends on two parameters:

1. **Epsilon (ε)** The radius around a point used to define its neighborhood.

$$N_\varepsilon(p) = \{q \in D \mid \text{dist}(p, q) \leq \varepsilon\}$$

where:

- p = query point
- q = neighboring point
- D = dataset

2. **Minimum Points (MinPts)** The minimum number of points required within the ε -neighborhood for a point to be considered a core point.

Typical guideline:

$$\text{MinPts} \geq d + 1$$

where d is the feature dimension.

Types of Points

1. **Core Point:**

$$|N_\varepsilon(p)| \geq \text{MinPts}$$

2. **Border Point:**

A point that belongs to the neighborhood of a core point but does not satisfy the core point condition.

3. **Noise (Outlier) Point:**

A point that is neither a core point nor a border point.

Mathematical Representation

Let

$$X = \{x_1, x_2, \dots, x_n\}$$

represent the dataset.

Step 1: Distance Function Typically Euclidean distance is used:

$$d(x_i, x_j) = \sqrt{\sum_{k=1}^d (x_{ik} - x_{jk})^2}$$

Step 2: Epsilon Neighborhood For a point p ,

$$N_\varepsilon(p) = \{q \in X \mid d(p, q) \leq \varepsilon\}$$

Step 3: Density Reachability A point q is directly density reachable from point p if:

$$q \in N_\varepsilon(p)$$

and

$$|N_\varepsilon(p)| \geq \text{MinPts}$$

Density Connectivity Two points p and q are density connected if there exists a sequence of density-reachable points connecting them.

Working Algorithm

1. Select values for ε and MinPts.
2. Choose an unvisited point.
3. Find all points within its ε -neighborhood.
4. If the point is a core point, create a cluster.
5. Expand the cluster using density-reachable points.
6. Mark isolated points as noise.
7. Repeat until all points are visited.

Graph Representation

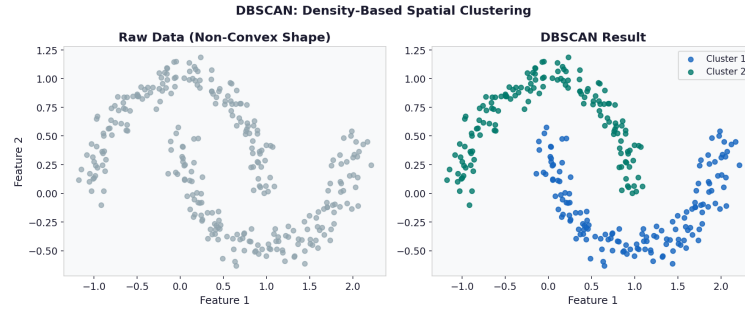


Figure 5.3: DBSCAN detects non-convex clusters (moons) and marks noise points (red $\times$)

Illustrative density-based clustering pattern:

Selecting Parameters

i. Choosing Epsilon (ε)

The most common approach is the k-distance graph.

1. Compute distance to the k-th nearest neighbor.
2. Sort distances in ascending order.
3. Plot the distances.
4. Select ε at the knee (elbow) point.

The chosen ε corresponds to the point where the curve begins to rise sharply.

ii. Choosing MinPts

General guidelines:

$$MinPts = d + 1$$

or

$$MinPts = 2d$$

where d is the feature dimension.

Higher-dimensional datasets generally require larger MinPts values.

Advantages of DBSCAN

- Does not require specifying the number of clusters.
- Can discover arbitrarily shaped clusters.
- Detects outliers automatically.
- Robust against noise.
- Effective for spatial datasets.

Limitations of DBSCAN

- Sensitive to parameter selection.
- Performance decreases in high-dimensional spaces.
- Struggles with clusters of varying densities.
- Choice of ε may be difficult.

5.2.3 Gaussian Mixture Model (GMM)

Gaussian Mixture Model (GMM) is a probabilistic clustering algorithm that assumes the dataset is generated from a mixture of multiple Gaussian distributions. Unlike K-Means, which assigns each point to exactly one cluster, GMM provides soft clustering, where each data point belongs to every cluster with a certain probability.

Data Assign to Cluster

Given a dataset

$$X = \{x_1, x_2, \dots, x_n\}$$

and K Gaussian components, the probability that point x_i belongs to cluster k is:

$$P(z_i = k | x_i) = \frac{\pi_k \mathcal{N}(x_i | \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(x_i | \mu_j, \Sigma_j)}$$

where:

- π_k = mixing coefficient of cluster k
- μ_k = mean of cluster k
- Σ_k = covariance matrix of cluster k
- $\mathcal{N}(x_i | \mu_k, \Sigma_k)$ = Gaussian density function

This probability is called the **responsibility** of cluster k for point x_i .

Hard Assignment After computing probabilities, a point can be assigned to the cluster having the highest probability:

$$Cluster(x_i) = \arg \max_k P(z_i = k | x_i)$$

Soft Assignment Unlike K-Means:

$$x_i = 0.8C_1 + 0.15C_2 + 0.05C_3$$

meaning the point belongs to multiple clusters with different probabilities.

Gaussian Distribution

GMM assumes that each cluster follows a Gaussian (Normal) Distribution.

For a d-dimensional feature vector:

$$\mathcal{N}(x|\mu, \Sigma) = \frac{1}{(2\pi)^{d/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x - \mu)^T \Sigma^{-1}(x - \mu)\right)$$

where:

- μ = Mean vector
- Σ = Covariance matrix
- $|\Sigma|$ = Determinant of covariance matrix

Mixture Model The complete probability distribution is:

$$P(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x|\mu_k, \Sigma_k)$$

subject to

$$\sum_{k=1}^K \pi_k = 1$$

Parameters Learned GMM learns:

- Mean (μ_k)
- Covariance (Σ_k)
- Mixing coefficient (π_k)

Loss Function (Log-Likelihood)

Unlike K-Means which minimizes WCSS, GMM maximizes the likelihood of observing the data.

$$L(\theta) = \prod_{i=1}^n \sum_{k=1}^K \pi_k \mathcal{N}(x_i | \mu_k, \Sigma_k)$$

To simplify optimization, we use Log-Likelihood:

$$\log L(\theta) = \sum_{i=1}^n \log \left(\sum_{k=1}^K \pi_k \mathcal{N}(x_i | \mu_k, \Sigma_k) \right)$$

The objective is:

$$\max \log L(\theta)$$

Expectation Maximization (EM) Algorithm

GMM parameters are estimated using the Expectation Maximization (EM) algorithm.

Step 1: Initialization Initialize:

$$\mu_k, \Sigma_k, \pi_k$$

randomly or using K-Means.

Step 2: Expectation (E-Step) Compute responsibilities:

$$\gamma(z_{ik}) = P(z_i = k | x_i)$$

$$\gamma(z_{ik}) = \frac{\pi_k \mathcal{N}(x_i | \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(x_i | \mu_j, \Sigma_j)}$$

Step 3: Maximization (M-Step) Update parameters:

$$N_k = \sum_{i=1}^n \gamma(z_{ik})$$

Mean:

$$\mu_k = \frac{1}{N_k} \sum_{i=1}^n \gamma(z_{ik}) x_i$$

Covariance:

$$\Sigma_k = \frac{1}{N_k} \sum_{i=1}^n \gamma(z_{ik}) (x_i - \mu_k)(x_i - \mu_k)^T$$

Mixing coefficient:

$$\pi_k = \frac{N_k}{n}$$

Step 4 Repeat E-Step and M-Step until convergence.

Graph Representation

The following figure illustrates Gaussian clusters and probability density contours.

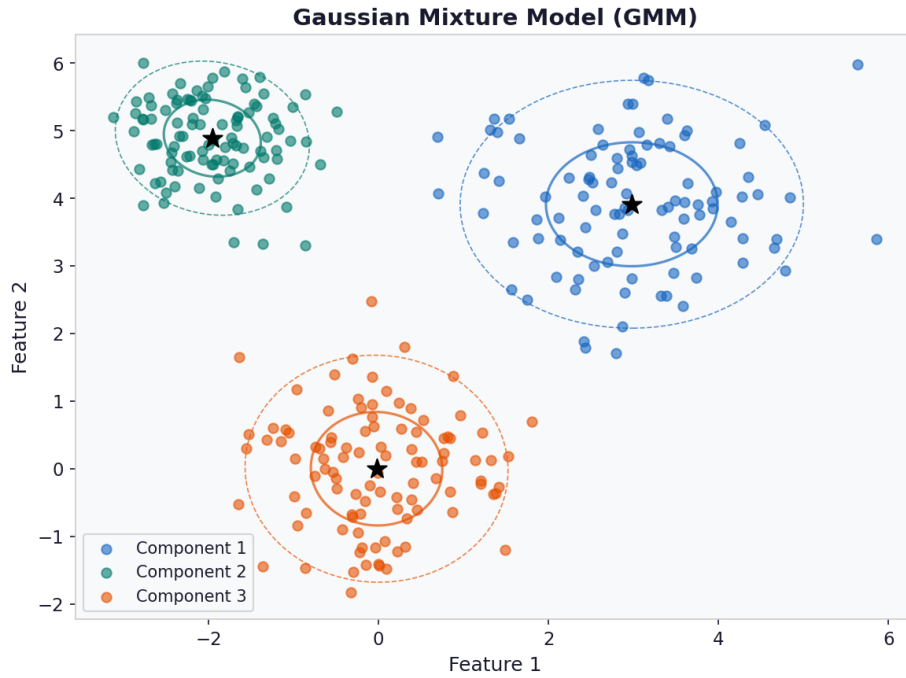


Figure 5.4: GMM with 3 components - solid/dashed ellipses show 1σ and 2σ contours; $\star$ = means

Drawbacks

i. Sensitive to Initialization

GMM can converge to a local optimum depending on the initial parameter values.

Problems:

- Different initial means may produce different clusters.
- Slow convergence.
- May converge to poor solutions.

Solution: Expectation Maximization (EM) EM iteratively improves parameter estimates and increases log-likelihood until convergence.

Advantages of EM:

- Stable parameter estimation.
- Better convergence.
- Probabilistic cluster assignment.

ii. Assumption of Gaussian Distribution

GMM assumes that each cluster follows a Gaussian distribution.

Limitations:

- Poor performance for non-Gaussian data.
- Sensitive to outliers.
- Struggles with highly irregular cluster shapes.

Advantages of GMM

- Soft clustering capability.
- Captures uncertainty in cluster assignments.
- Models elliptical clusters.
- More flexible than K-Means.
- Provides probabilistic interpretation.

5.2.4 Hierarchical Clustering

Hierarchical Clustering is an unsupervised machine learning algorithm that builds a hierarchy of clusters. Unlike K-Means, it does not require the number of clusters to be specified beforehand. The output is typically represented using a dendrogram, which illustrates how clusters are merged or split at different levels.

Types

i. Agglomerative Clustering (Bottom-Up)

Agglomerative clustering starts with each data point as an individual cluster and repeatedly merges the closest clusters until only one cluster remains.

Algorithm

1. Treat each data point as a separate cluster.
2. Compute pairwise distances between all clusters.
3. Merge the two closest clusters.
4. Update the distance matrix.
5. Repeat Steps 2–4 until all points belong to a single cluster.

Mathematical Representation Initially,

$$C_i = \{x_i\}, \quad i = 1, 2, \dots, n$$

At each iteration:

$$(C_p, C_q) = \arg \min_{i,j} D(C_i, C_j)$$

where $D(C_i, C_j)$ is the linkage distance between clusters.

ii. Divisive Clustering (Top-Down)

Divisive clustering starts with all data points in a single cluster and recursively splits clusters until each cluster contains a single point or a stopping criterion is reached.

Algorithm

1. Place all data points in one cluster.
2. Identify the least homogeneous cluster.
3. Split the cluster into two subclusters.
4. Repeat until the desired number of clusters is obtained.

Mathematical Representation Initially:

$$C = X$$

where X is the complete dataset.

At each iteration:

$$C \rightarrow C_1 \cup C_2$$

such that the within-cluster variance is minimized.

Distance Metrics

Distance metrics determine the similarity between data points.

Euclidean Distance Most commonly used distance metric.

$$d(x_i, x_j) = \sqrt{\sum_{k=1}^d (x_{ik} - x_{jk})^2}$$

Manhattan Distance

$$d(x_i, x_j) = \sum_{k=1}^d |x_{ik} - x_{jk}|$$

Minkowski Distance

$$d(x_i, x_j) = \left(\sum_{k=1}^d |x_{ik} - x_{jk}|^p \right)^{1/p}$$

Cosine Distance

$$d(x_i, x_j) = 1 - \frac{x_i \cdot x_j}{\|x_i\| \|x_j\|}$$

Linkage Methods

A linkage method determines how the distance between clusters is calculated.

Let

$$C_i = \{x_1, x_2, \dots\}$$

and

$$C_j = \{y_1, y_2, \dots\}$$

represent two clusters.

i. Single Linkage

Single linkage uses the minimum distance between points from two clusters.

$$D_{\text{single}}(C_i, C_j) = \min_{x \in C_i, y \in C_j} d(x, y)$$

Characteristics

- Produces elongated clusters.
- Sensitive to noise.
- Suffers from chaining effect.

ii. Complete Linkage

Complete linkage uses the maximum distance between points from two clusters.

$$D_{\text{complete}}(C_i, C_j) = \max_{x \in C_i, y \in C_j} d(x, y)$$

Characteristics

- Produces compact clusters.
- Less sensitive to noise.
- Reduces chaining effect.

iii. Average Linkage

Average linkage uses the average pairwise distance between all points in two clusters.

$$D_{\text{average}}(C_i, C_j) = \frac{1}{|C_i||C_j|} \sum_{x \in C_i} \sum_{y \in C_j} d(x, y)$$

Characteristics

- Balances single and complete linkage.
- Produces relatively stable clusters.
- Commonly used in practical applications.

Dendrogram Representation

A dendrogram is a tree-like structure that visualizes the hierarchical clustering process.

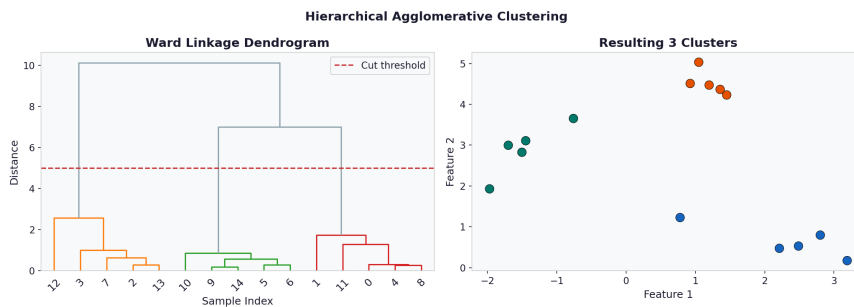


Figure 5.5: Ward dendrogram (left) and resulting 3 clusters (right). Red dashed line = cut threshold.

Advantages

- No need to specify the number of clusters initially.
- Produces interpretable dendrograms.
- Works well for small and medium-sized datasets.
- Can discover nested cluster structures.

Limitations

- Computationally expensive for large datasets.
- Sensitive to noise and outliers.
- Once clusters are merged or split, the decision cannot be undone.
- Choice of linkage method affects clustering results.

Comparison of Linkage Methods

Method	Formula Basis	Cluster Shape	Noise Sensitivity
Single Linkage	Minimum Distance	Elongated	High
Complete Linkage	Maximum Distance	Compact	Low
Average Linkage	Mean Distance	Balanced	Medium

Table 5.2: Comparison of Linkage Methods

5.3 Dimensionality Reduction

This models used to reduce the number of input features(dimensions) in a dataset, there are too many features, which may cause: High computational cost, overfitting, difficult visualization.

Dimensionality Reduction transforms high-dimensional data into lower-dimensions.

5.3.1 Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is a dimensionality reduction technique that transforms a high-dimensional dataset into a new set of orthogonal variables called *principal components*. These components capture the maximum variance present in the data while reducing redundancy among features. commonly used for linear projection.

Mathematical Representation

Given a centered dataset

$$X = x_1, x_2, \dots, x_n, \quad x_i \in \mathbb{R}^d$$

the covariance matrix is computed as

$$C = \frac{1}{n-1} X^T X$$

where:

- (X) is the centered data matrix,
- (n) is the number of samples,
- (C) is the covariance matrix.

The principal components are obtained by solving the eigenvalue problem:

$$Cv = \lambda v$$

where:

- (v) = eigenvector (direction of new feature),
- (λ) = eigenvalue (importance of that feature).

The eigenvectors corresponding to the largest eigenvalues are selected as the principal components because they capture the maximum variance in the dataset.

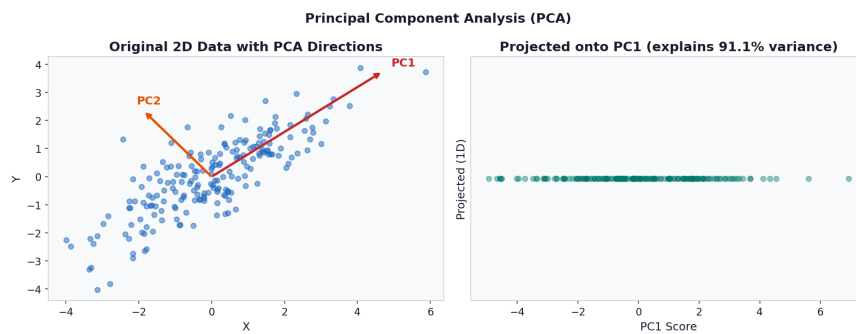


Figure 5.6: PCA principal directions (left) and projection onto PC1 (right).

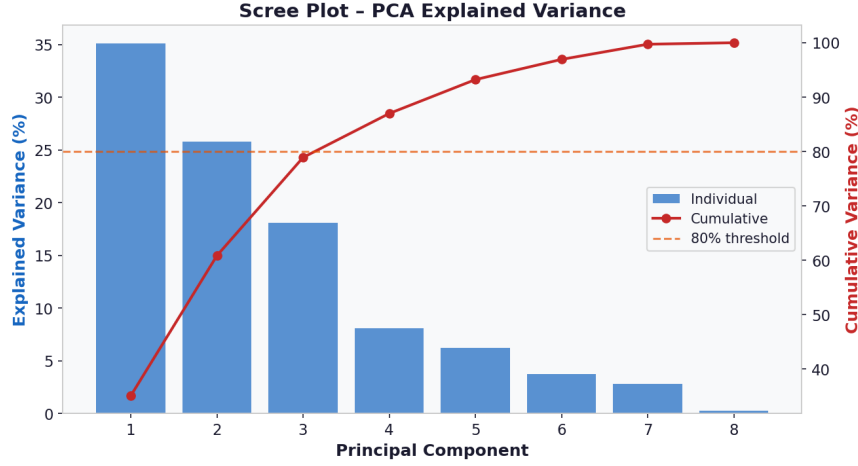


Figure 5.7: Scree Plot: individuals and cumulative explained variance per component

5.3.2 t-Distributed Stochastic Neighbor Embedding (t-SNE)

t-Distributed Stochastic Neighbor Embedding (t-SNE) is a non-linear dimensionality reduction technique primarily used for visualization of high-dimensional data in two-dimensional (2D) or three-dimensional (3D) space. Unlike PCA, which preserves global variance, t-SNE focuses on preserving local neighborhood structures, making it particularly effective for visualizing clusters and complex data patterns.

Mathematical Representation

For two high-dimensional data points x_i and x_j , the similarity is defined as

$$p_{j|i} = \frac{\exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma_i^2}\right)}{\sum_{k \neq i} \exp\left(-\frac{\|x_i - x_k\|^2}{2\sigma_i^2}\right)}$$

The joint probability in the original space is

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}$$

In the low-dimensional embedding space, similarities are computed using the Student's t-distribution:

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq i} (1 + \|y_i - y_k\|^2)^{-1}}$$

The objective is to minimize the Kullback-Leibler (KL) divergence between the two distributions:

$$KL(P \parallel Q) = \sum_i \sum_j p_{ij} \log\left(\frac{p_{ij}}{q_{ij}}\right)$$

Applications

t-SNE is commonly used for:

- Visualization of high-dimensional datasets.
- Image embedding visualization.
- Cluster analysis and cluster visualization.
- Natural Language Processing (word embeddings).
- Non-linear dimensionality reduction.
- Feature exploration and pattern discovery.

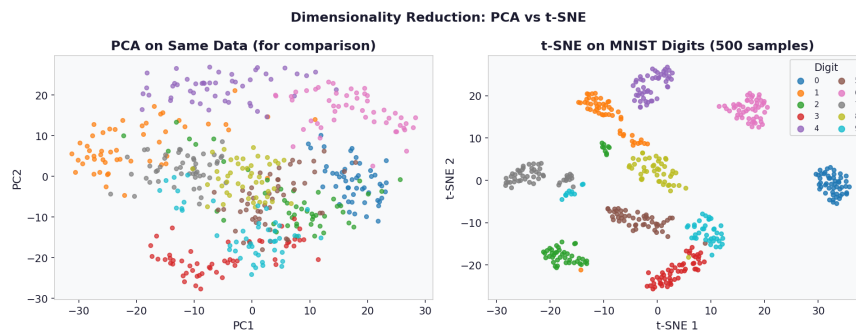


Figure 5.8: PCA (left) vs t-SNE (right) on 500 MNIST digit samples — t-SNE reveals cleaner clusters.

5.4 Generative Models

5.4.1 Autoencoders (AE)

Autoencoders (AEs) are neural networks designed to learn efficient representations of data by compressing the input into a lower-dimensional latent space and then reconstructing the original input. They are widely used for dimensionality reduction, feature learning, denoising, and anomaly detection.

Applications

- Remove noise from data and improve data quality (Denoising Autoencoders).
- Detect unusual patterns or anomalies in data.
- Extract meaningful features for downstream machine learning tasks.
- Reduce dimensionality while preserving important information.

Architecture

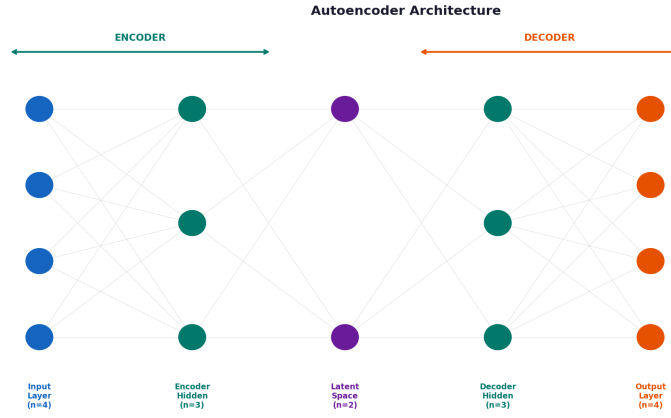


Figure 5.9: Autoencoder architecture: the Encoder compresses the input into a latent representation (z), and the Decoder reconstructs the output ($\hat{x}$).

Components

- **Encoder:** Compresses the input data into a lower-dimensional latent representation.
- **Bottleneck (Latent Space):** Stores the compressed representation and captures the most important features of the input data.
- **Decoder:** Reconstructs the original input from the latent representation.

Anomaly Detection Using Autoencoders

For anomaly detection, the autoencoder is trained using only normal data. During inference, normal samples are reconstructed accurately because the model has learned their patterns. However, anomalous samples differ significantly from the training distribution and therefore produce larger reconstruction errors.

A high reconstruction error indicates that the input may be an anomaly.

Reconstruction Loss Function

The reconstruction error is commonly measured using Mean Squared Error (MSE):

$$L = \|X - \hat{X}\|^2$$

where:

- X = original input,
- $\hat{X}$ = reconstructed output,
- L = reconstruction loss.

For a dataset containing n samples, the MSE loss is:

$$L = \frac{1}{n} \sum_{i=1}^n (X_i - \hat{X}_i)^2$$

The objective of training is to minimize the reconstruction loss:

$$\min L$$

5.4.2 Variational Autoencoders (VAE)

Variational Autoencoders (VAEs) are probabilistic generative models that learn a continuous latent representation of data. Unlike traditional Autoencoders, which encode an input into a fixed latent vector, VAEs learn a probability distribution in the latent space. This enables the generation of new data samples by sampling from the learned latent distribution.

VAEs are widely used for data generation, image synthesis, anomaly detection, and representation learning.

Components

- **Encoder:** Maps the input (X) to a latent probability distribution.
- **Mean Vector (μ):** Represents the center of the latent distribution.
- **Standard Deviation (σ):** Represents the spread of the latent distribution.
- **Latent Variable (z):** Sampled from the learned distribution.
- **Decoder:** Reconstructs the original input from z .

Mathematical Representation

Unlike a traditional Autoencoder, a Variational Autoencoder (VAE) learns a probability distribution in the latent space rather than a fixed latent vector.

Given an input sample x , the encoder maps it to a latent distribution:

$$q_\phi(z | x) = \mathcal{N}(\mu, \sigma^2)$$

where:

- x = input data,
- z = latent variable,
- μ = mean vector,
- σ^2 = variance vector,

- $q_\phi(z \mid x)$ = approximate posterior distribution.

The latent variable is sampled from the learned distribution:

$$z \sim \mathcal{N}(\mu, \sigma^2)$$

To allow gradient-based optimization, VAE uses the reparameterization trick:

$$z = \mu + \sigma \odot \epsilon$$

where

$$\epsilon \sim \mathcal{N}(0, 1)$$

and $\odot$ denotes element-wise multiplication.

The decoder reconstructs the input from the latent representation:

$$p_\theta(x \mid z)$$

where $p_\theta(x \mid z)$ represents the probability of reconstructing x from z .

The objective of the VAE is to learn a latent space that follows a standard normal distribution:

$$p(z) = \mathcal{N}(0, 1)$$

while accurately reconstructing the input data.

Applications

- Image generation.
- Data synthesis.
- Anomaly detection.
- Feature extraction.
- Dimensionality reduction.
- Drug discovery and bioinformatics.

Chapter 6

Model Evaluation

Model evaluation is the process of assessing the performance of a machine learning model. Different metrics are used for supervised and unsupervised learning tasks.

6.1 Evaluation of Supervised Learning Models

6.1.1 Confusion Matrix

	Predicted Positive	Predicted Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

Table 6.1: Confusion Matrix

6.1.2 Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

6.1.3 Precision

$$\text{Precision} = \frac{TP}{TP + FP}$$

6.1.4 Recall

$$\text{Recall} = \frac{TP}{TP + FN}$$

6.1.5 F1-Score

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

6.1.6 Specificity

$$\text{Specificity} = \frac{TN}{TN + FP}$$

6.1.7 Mean Squared Error (MSE)

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

where:

- y_i = actual value
- $\hat{y}_i$ = predicted value
- n = number of samples

6.1.8 Root Mean Squared Error (RMSE)

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

6.1.9 Mean Absolute Error (MAE)

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

6.1.10 Coefficient of Determination (R^2)

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

where $\bar{y}$ is the mean of actual values.

6.2 Evaluation of Unsupervised Learning Models

6.2.1 Silhouette Score

Let

$a(i)$ = Average distance of point i within its own cluster

and

$b(i)$ = Minimum average distance of point i to another cluster

Then,

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

where

$$-1 \leq s(i) \leq 1$$

- $s(i) \approx 1$: Good clustering
- $s(i) \approx 0$: Overlapping clusters
- $s(i) < 0$: Possible misclassification

6.2.2 Davies-Bouldin Index (DBI)

$$DBI = \frac{1}{K} \sum_{i=1}^K \max_{j \neq i} \left(\frac{S_i + S_j}{M_{ij}} \right)$$

where:

- S_i = intra-cluster distance
- M_{ij} = centroid distance between clusters

Lower values indicate better clustering.

6.2.3 Calinski-Harabasz Index (CHI)

$$CHI = \frac{B_K}{W_K} \times \frac{n - K}{K - 1}$$

where:

- B_K = between-cluster dispersion
- W_K = within-cluster dispersion
- n = total samples
- K = number of clusters

Higher values indicate better clustering.

6.2.4 Within Cluster Sum of Squares (WCSS)

$$WCSS = \sum_{k=1}^K \sum_{x_i \in C_k} (x_i - \mu_k)^2$$

where:

- C_k = cluster k
- μ_k = centroid of cluster k

Lower values indicate compact clusters.

6.2.5 Reconstruction Error

Used in Autoencoders and Variational Autoencoders.

$$L = \sum_{i=1}^n (X_i - \hat{X}_i)^2$$

where:

- X_i = original input
- $\hat{X}_i$ = reconstructed input

Lower reconstruction error indicates better reconstruction quality.

Chapter 7

Machine Learning Pipelines

A Machine Learning (ML) Pipeline is a structured workflow that automates the sequence of steps required to build, evaluate, deploy, and maintain machine learning models. Pipelines improve reproducibility, scalability, and efficiency by organizing the machine learning process into interconnected stages.

7.1 Overview of Machine Learning Pipeline

A typical machine learning pipeline consists of the following stages:

1. Data Collection
2. Data Preprocessing
3. Feature Engineering
4. Data Splitting
5. Model Training
6. Model Evaluation
7. Hyperparameter Tuning
8. Model Deployment
9. Monitoring and Maintenance

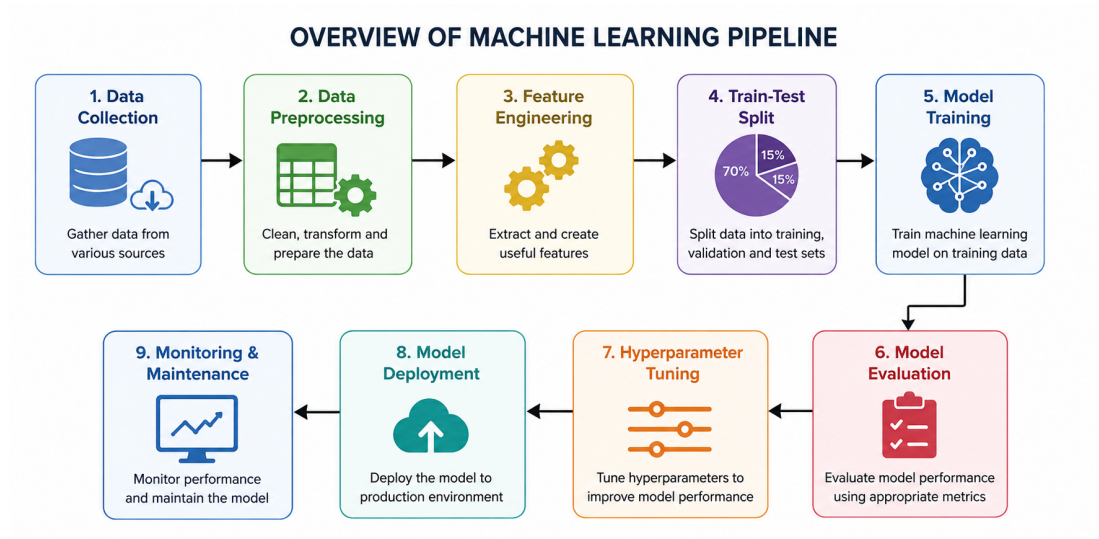


Figure 7.1: Overview of a Machine Learning Pipeline

7.2 Data Collection

Data collection is the first stage of the pipeline. The quality of data directly affects model performance.

7.2.1 Sources of Data

- Databases
- APIs
- IoT Sensors
- Web Scraping
- Public Datasets
- User-generated Data

7.2.2 Objectives

- Gather relevant data
- Ensure sufficient data quantity
- Maintain data quality

7.3 Data Preprocessing

Raw data often contains missing values, noise, duplicate records, and inconsistencies. Data preprocessing transforms raw data into a usable format.

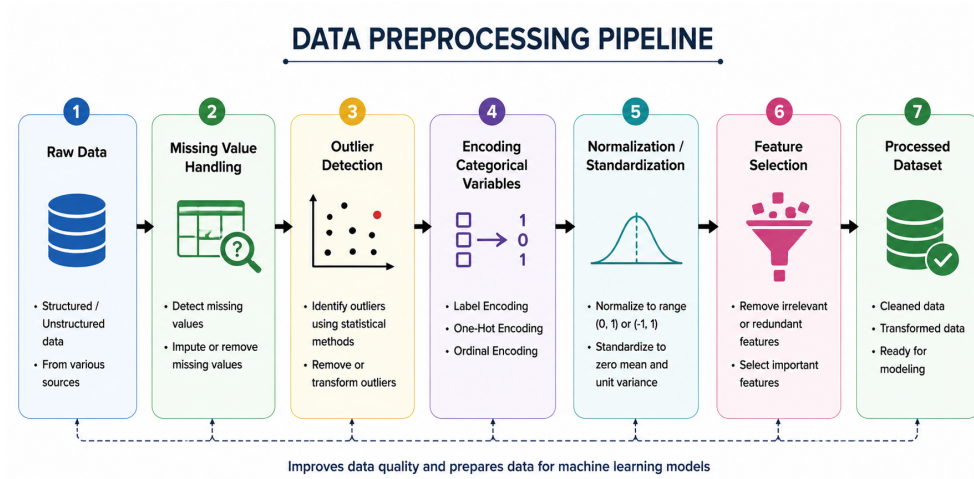


Figure 7.2: Data Preprocessing Pipeline

7.3.1 Handling Missing Values

Common methods:

- Mean Imputation
- Median Imputation
- Mode Imputation
- Deletion

Mean:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

7.3.2 Normalization

Normalization scales features into a fixed range.

Min-Max Scaling:

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

7.3.3 Standardization

Standardization transforms data into zero mean and unit variance.

$$z = \frac{x - \mu}{\sigma}$$

where:

- μ = mean
- σ = standard deviation

7.4 Feature Engineering

Feature Engineering involves creating meaningful features that improve model performance.

7.4.1 Techniques

- Feature Selection
- Feature Extraction
- Dimensionality Reduction
- Encoding Categorical Variables

7.4.2 Principal Component Analysis (PCA)

Data covariance matrix:

$$C = \frac{1}{n-1} X^T X$$

Eigenvalue equation:

$$Cv = \lambda v$$

where:

- v = eigenvector
- λ = eigenvalue

7.5 Data Splitting

The dataset is divided into training, validation, and testing sets.

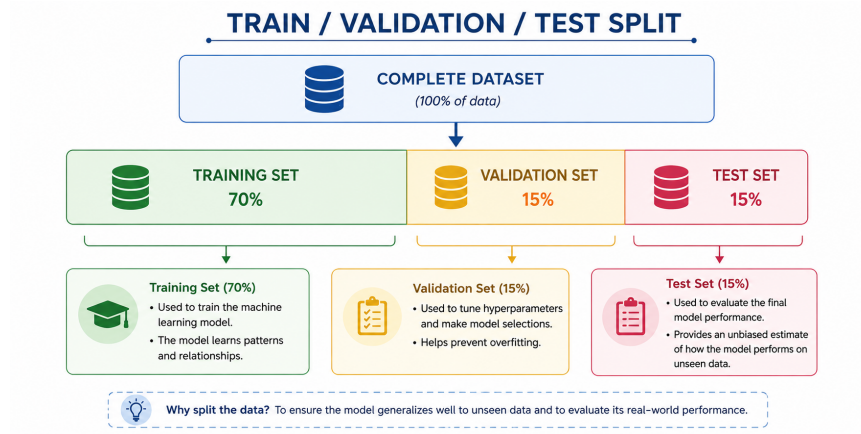


Figure 7.3: Train, Validation and Test Split

7.5.1 Typical Split Ratios

- 70% Training
- 15% Validation
- 15% Testing

7.6 Model Training

Training is the process of learning patterns from data.

Given:

$$D = \{(x_i, y_i)\}_{i=1}^n$$

the objective is to learn:

$$f(x) \approx y$$

7.6.1 Loss Function

Mean Squared Error (MSE):

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

7.7 Model Evaluation

The trained model is evaluated using appropriate metrics.

7.7.1 Classification Metrics

Accuracy:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall:

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1 Score:

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

7.7.2 Regression Metrics

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

7.8 Hyperparameter Tuning

Hyperparameters are parameters configured before training.

Examples:

- Learning Rate
- Number of Trees
- Number of Neurons
- Batch Size

7.8.1 Grid Search

Tests all parameter combinations.

7.8.2 Random Search

Randomly samples parameter combinations.

7.9 Model Deployment

After successful evaluation, the model is deployed for real-world usage.

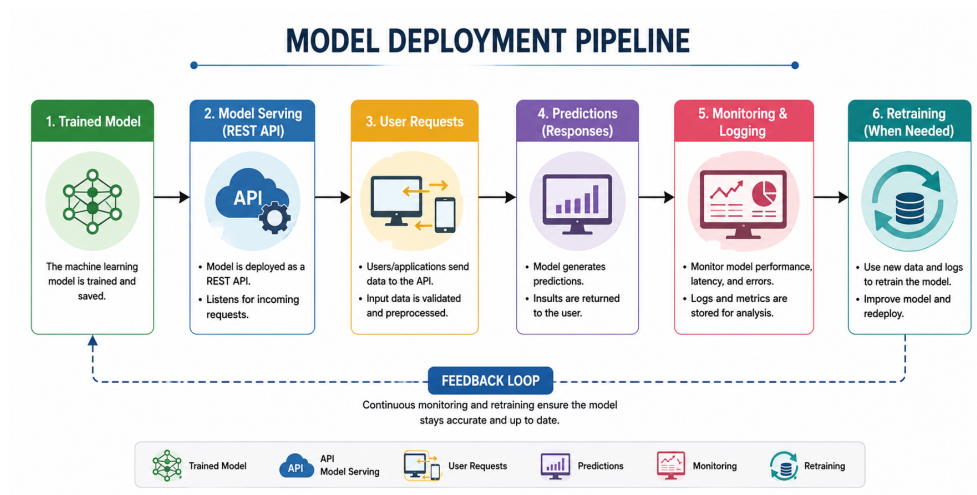


Figure 7.4: Machine Learning Model Deployment Pipeline

7.9.1 Deployment Methods

- REST APIs
- Cloud Services
- Mobile Applications
- Edge Devices

7.10 Monitoring and Maintenance

Deployed models require continuous monitoring.

7.10.1 Monitoring Metrics

- Accuracy Drift
- Data Drift
- Concept Drift
- Latency

7.10.2 Retraining

Models should be retrained periodically when performance degrades.

7.11 Advantages of ML Pipelines

- Automation of repetitive tasks
- Improved reproducibility
- Better scalability
- Faster deployment
- Easier maintenance

Chapter 8

Working with Text Data

Chapter 9

Deep Learning Basics

Chapter 10

Case Studies and Applications